

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA IN INGEGNERIA INFORMATICA
CORSO DI INGEGNERIA DEL SOFTWARE
PROF. A.R. FASOLINO - A.A. 2025-26...



**TEMPLATE PER IL PROGETTO
DI INGEGNERIA DEL SOFTWARE**

**“PIATTAFORMA DI GESTIONE E
CONDIVISIONE DI MATERIALE DIDATTICO”**

INDICE

1. SPECIFICHE INFORMALI	3
2. ANALISI E SPECIFICA DEI REQUISITI	5
2.1 ANALISI NOMI-VERBI	5
2.2 REVISIONE DEI REQUISITI	7
2.3 GLOSSARIO DEI TERMINI	8
2.4 CLASSIFICAZIONE DEI REQUISITI	8
2.4.1 Requisiti funzionali.....	8
2.4.2 Requisiti sui dati	9
2.4.3 Vincoli / Altri requisiti	11
2.5 MODELLAZIONE DEI CASI D'USO	12
2.5.1 Attori e casi d'uso.....	12
2.5.2 Diagramma dei casi d'uso.....	13
2.5.3 Scenari.....	13
2.6 DIAGRAMMA DELLE CLASSI.....	15
2.6.1 Responsabilità	<i>Errore. Il segnalibro non è definito.</i>
2.7 DIAGRAMMI DI SEQUENZA	18
2.7.1 Registrazione	<i>Errore. Il segnalibro non è definito.</i>
2.7.2 Accesso.....	<i>Errore. Il segnalibro non è definito.</i>
2.7.3 AggiungiAuto	<i>Errore. Il segnalibro non è definito.</i>
3. PIANO DI TEST FUNZIONALE	21
3.1 REGISTRAZIONE.....	ERRORE. IL SEGNALIBRO NON È DEFINITO.
4. PROGETTAZIONE	25
4.1 DIAGRAMMA DELLE CLASSI.....	25
4.1.1 Traduzione classi ed associazioni.....	26
4.1.2 Pattern BCED.....	27
4.1.2.1 Package Boundary	28
4.1.2.2 Package Controller	29
4.1.2.3 Package Entity	30
4.1.2.4 Package Database.....	31
4.2 DIAGRAMMI DI SEQUENZA	32
4.2.1 Registrazione	<i>Errore. Il segnalibro non è definito.</i>
4.2.2 Accesso.....	<i>Errore. Il segnalibro non è definito.</i>
5. IMPLEMENTAZIONE	36
5.1 PACKAGE DATABASE	36
5.2 PACKAGE ENTITY	37
5.3 PACKAGE CONTROLLER.....	37
5.4 PACKAGE BOUNDARY	37
5.5 PACKAGE DTO	38
5.6 DIAGRAMMA DI DEPLOYMENT	38
6. TESTING	39
6.1 TEST STRUTTURALE.....	ERRORE. IL SEGNALIBRO NON È DEFINITO.
6.1.1 Complessità ciclomatica	<i>Errore. Il segnalibro non è definito.</i>
6.1.1.1 inserisciAutoModifiche – GestioneParcoAuto.....	<i>Errore. Il segnalibro non è definito.</i>
6.2 JUNIT – TEST DI UNITÀ.....	41
6.3 TEST FUNZIONALE.....	43

1. Specifiche informali

Si intende sviluppare un sistema software per la gestione e la condivisione di materiale didattico pubblicato dai docenti e consultato dagli studenti.

Il sistema consente l'accesso di docenti e studenti a una piattaforma digitale mediante autenticazione tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo (studente o docente), oltre a fornire nome, cognome ed indirizzo email istituzionale.

Ogni docente ha la possibilità di creare e gestire uno o più corsi. Ciascun corso è identificato da un titolo, una descrizione, un codice univoco e, opzionalmente, dall'anno accademico di riferimento. L'iscrizione di uno studente a un corso può avvenire in due modalità: il docente può iscrivere direttamente lo studente al corso, oppure lo studente stesso può iscriversi autonomamente inserendo il codice univoco del corso fornito dal docente.

All'interno di ogni corso, il docente può pubblicare materiale didattico di vario tipo mediante un'apposita interfaccia grafica. Per ciascun contenuto pubblicato devono essere specificati almeno un titolo, una breve descrizione, una categoria e la data di pubblicazione. Le categorie possono comprendere, ad esempio, slide, dispense, esercizi, soluzioni, avvisi o materiale integrativo. Per semplicità, si può supporre che ogni contenuto appartenga a una sola categoria.

Il docente può inoltre organizzare i materiali in sezioni o moduli didattici, ciascuno identificato da un titolo. Deve essere possibile associare ogni contenuto a una determinata sezione del corso, in modo da agevolarne la consultazione. I materiali possono essere resi visibili immediatamente agli studenti oppure mantenuti temporaneamente non pubblicati fino a una successiva attivazione da parte del docente.

Ogni studente autenticato dispone di una propria interfaccia nella quale può visualizzare l'elenco dei corsi a cui è iscritto. Selezionando un corso, può consultare l'elenco completo dei materiali didattici disponibili, eventualmente filtrandoli per categoria, data o sezione. Deve inoltre essere possibile accedere al dettaglio di ciascun contenuto e visualizzarne le informazioni principali.

Il sistema mantiene traccia dei materiali pubblicati per ciascun corso e consente al docente di modificarne le informazioni o rimuoverli. Per semplicità, si può supporre che un materiale eliminato non sia più visibile agli studenti. Il docente può inoltre visualizzare l'elenco completo dei contenuti pubblicati per ciascun corso e monitorarne la distribuzione per categoria.

Ogni studente ha accesso a una sezione personale in cui può consultare i corsi seguiti e i materiali recentemente pubblicati. Il docente, attraverso un'interfaccia dedicata, può monitorare l'andamento complessivo dei propri corsi, visualizzando il numero totale di materiali pubblicati, il numero di studenti iscritti a ciascun corso e le categorie di contenuti maggiormente utilizzate.

Il sistema può includere una funzione di ricerca dei materiali per titolo, descrizione o categoria. Deve inoltre essere possibile visualizzare il profilo del docente associato a ciascun corso, comprensivo almeno del nome e dell'elenco dei corsi gestiti.

La piattaforma dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli

studenti della pubblicazione di nuovo materiale o dell'inserimento di eventuali avvisi da parte del docente. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni salvate e una corretta gestione dei permessi tra docenti e studenti.

2. Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Il sistema consente l'accesso di docenti e studenti a una piattaforma digitale mediante autenticazione tramite credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo (studente o docente), oltre a fornire nome, cognome ed indirizzo email istituzionale.

Ogni docente ha la possibilità di creare e gestire uno o più corsi. Ciascun corso è identificato da un titolo, una descrizione, un codice univoco e, opzionalmente, dall'anno accademico di riferimento. L'iscrizione di uno studente a un corso può avvenire in due modalità: il docente può iscrivere direttamente lo studente al corso, oppure lo studente stesso può iscriversi autonomamente inserendo il codice univoco del corso fornito dal docente.

All'interno di ogni corso, il docente può pubblicare materiale didattico di vario tipo mediante un'apposita interfaccia grafica. Per ciascun contenuto pubblicato devono essere specificati almeno un titolo, una breve descrizione, una categoria e la data di pubblicazione. Le categorie possono comprendere, ad esempio, slide, dispense, esercizi, soluzioni, avvisi o materiale integrativo. Per semplicità, si può supporre che ogni contenuto appartenga a una sola categoria.

Il docente può inoltre organizzare i materiali in sezioni o moduli didattici, ciascuno identificato da un titolo. Deve essere possibile associare ogni contenuto a una determinata sezione del corso, in modo da agevolarne la consultazione. I materiali possono essere resi visibili immediatamente agli studenti oppure mantenuti temporaneamente non pubblicati fino a una successiva attivazione da parte del docente.

Ogni studente autenticato dispone di una propria interfaccia nella quale può visualizzare l'elenco dei corsi a cui è iscritto. Selezionando un corso, può consultare l'elenco completo dei materiali didattici disponibili, eventualmente filtrandoli per categoria, data o sezione. Deve inoltre essere possibile accedere al dettaglio di ciascun contenuto e visualizzarne le informazioni principali.

Il sistema mantiene traccia dei materiali pubblicati per ciascun corso e consente al docente di modificarne le informazioni o rimuoverli. Per semplicità, si può supporre che un materiale eliminato non sia più visibile agli studenti. Il docente può inoltre visualizzare l'elenco completo dei contenuti pubblicati per ciascun corso e monitorarne la distribuzione per categoria.

Ogni studente ha accesso a una sezione personale in cui può consultare i corsi seguiti e i materiali recentemente pubblicati. Il docente, attraverso un'interfaccia dedicata, può monitorare l'andamento complessivo dei propri corsi, visualizzando il numero totale di materiali pubblicati, il numero di studenti iscritti a ciascun corso e le categorie di contenuti maggiormente utilizzate.

Il sistema può includere una funzione di ricerca dei materiali per titolo, descrizione o categoria. Deve inoltre essere possibile visualizzare il profilo del docente associato a ciascun corso, comprensivo almeno del nome e dell'elenco dei corsi gestiti.

La piattaforma dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli

studenti della pubblicazione di nuovo materiale o dell'inserimento di eventuali **avvisi** da parte del docente. Il sistema dovrà inoltre garantire la protezione dei dati personali, la sicurezza delle informazioni salvate e una corretta gestione dei permessi tra docenti e studenti.

Materiale Didattico = Contenuto.

LEGENDA:

Classe

Attributo

Funzionalità

Attore

Classe-Attore

2.2 Revisione dei requisiti

1. *Il sistema deve offrire all'Utente una funzionalità per registrarsi, specificando ruolo, nome, cognome ed email istituzionale.*
2. *Il sistema deve offrire all'Utente una funzionalità per accedere mediante credenziali personali.*
3. *Di ogni Utente si vuole memorizzare nome, cognome, email istituzionale, ruolo e credenziali di accesso.*
4. *Il sistema deve offrire al Docente una funzionalità per creare un Corso.*
5. *Il sistema deve offrire al Docente una funzionalità per modificare i dati di un Corso di cui è titolare.*
6. *Di ogni Corso si vuole memorizzare titolo, descrizione, codice univoco, anno accademico (opzionale) e Docente titolare.*
7. *Il sistema deve offrire al Docente una funzionalità per iscrivere direttamente uno Studente a un Corso.*
8. *Il sistema deve offrire allo Studente una funzionalità per iscriversi autonomamente a un Corso inserendo il codice univoco.*
9. *Di ogni Iscrizione si vuole memorizzare lo Studente e il Corso associati.*
10. *Il sistema deve offrire al Docente una funzionalità per pubblicare un Contenuto didattico all'interno di un Corso.*
11. *Di ogni Contenuto si vuole memorizzare titolo, descrizione, categoria, data di pubblicazione e stato di pubblicazione.*
12. *Ogni Contenuto deve appartenere a una sola Categoria (slide, dispense, esercizi, soluzioni, avvisi o materiale integrativo).*
13. *Il sistema deve offrire al Docente una funzionalità per organizzare i Contenuti di un Corso in Sezioni.*
14. *Di ogni Sezione si vuole memorizzare titolo e Corso di appartenenza.*
15. *Il sistema deve offrire al Docente una funzionalità per associare un Contenuto a una Sezione del Corso.*
16. *Il sistema deve offrire al Docente una funzionalità per rendere visibile o mantenere non pubblicato un Contenuto, attivandolo successivamente.*
17. *Il sistema deve offrire al Docente una funzionalità per modificare un Contenuto pubblicato.*
18. *Il sistema deve offrire al Docente una funzionalità per rimuovere un Contenuto pubblicato.*
19. *Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Corsi a cui è iscritto.*
20. *Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Contenuti di un Corso.*
21. *Il sistema deve offrire allo Studente una funzionalità per filtrare i Contenuti di un Corso per categoria, data o Sezione.*
22. *Il sistema deve offrire allo Studente una funzionalità per visualizzare il dettaglio di un Contenuto.*
23. *Il sistema deve offrire al Docente una funzionalità per visualizzare l'elenco dei Contenuti pubblicati di un Corso.*
24. *Il sistema deve offrire al Docente una funzionalità per visualizzare la distribuzione dei Contenuti per Categoria in un Corso.*
25. *Il sistema deve offrire al Docente una funzionalità per monitorare l'andamento di un Corso, visualizzando numero di Contenuti pubblicati, numero di Studenti iscritti e Categorie più utilizzate.*
26. *Il sistema deve offrire all'Utente una funzionalità per cercare Contenuti per titolo, descrizione o categoria.*

27. Il sistema deve offrire all'Utente una funzionalità per visualizzare il profilo di un Docente, comprensivo di nome ed elenco dei Corsi gestiti.
28. Il sistema deve inviare una Notifica allo Studente quando viene pubblicato un nuovo Contenuto in un Corso a cui è iscritto.
29. Il sistema deve inviare una Notifica allo Studente quando il Docente pubblica un avviso nel Corso.

2.3 Glossario dei termini

[OPZIONALE] Riportare un glossario dei termini in una tabella termine-descrizione (significato)-eventuali sinonimi, come nel seguente esempio.

Termine	Descrizione	Sinonimi
Contenuto	Un'unità di materiale didattico pubblicata dal Docente all'interno di un Corso	Materiale didattico
Sezione	Modulo in cui il Docente organizza i Contenuti di un Corso per agevolarne la consultazione	Modulo didattico
Utente	Persona registrata alla piattaforma con ruolo Docente o Studente	
Iscrizione	Associazione tra uno Studente e un Corso a cui è abilitato ad accedere	
Categoria	Tipo di un Contenuto (slide, dispense, esercizi, soluzioni, avvisi, materiale integrativo)	
Notifica	Messaggio inviato allo Studente per informarlo della pubblicazione di nuovo materiale o di un avviso del Docente	

2.4 Classificazione dei requisiti

Classificare i requisiti (funzionali, sui dati, vincoli/altri requisiti), riportando le frasi dei requisiti (revisionati) che li esprimono (ciascuna frase deve esprimere un singolo requisito), numerandoli secondo una codifica (es.: RF01, RF02, RFn per i requisiti funzionali; RD01, RD02, RDm per i requisiti sui dati, RQ01, etc per i requisiti non funzionali di qualità, V01... per eventuali vincoli progettuali). Verificare che i requisiti siano chiari, completi e verificabili.

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RF01	Il sistema deve offrire all'Utente una funzionalità per registrarsi, specificando ruolo, nome, cognome ed email istituzionale	1
RF02	Il sistema deve offrire all'Utente una funzionalità per accedere mediante credenziali personali	2
RF03	Il sistema deve offrire al Docente una funzionalità per creare un Corso	4

RF04	Il sistema deve offrire al Docente una funzionalità per modificare i dati di un Corso di cui è titolare	5
RF05	Il sistema deve offrire al Docente una funzionalità per iscrivere direttamente uno Studente a un Corso	7
RF06	Il sistema deve offrire allo Studente una funzionalità per iscriversi autonomamente a un Corso inserendo il codice univoco	8
RF07	Il sistema deve offrire al Docente una funzionalità per pubblicare un Contenuto didattico all'interno di un Corso	10
RF08	Il sistema deve offrire al Docente una funzionalità per organizzare i Contenuti di un Corso in Sezioni	13
RF09	Il sistema deve offrire al Docente una funzionalità per associare un Contenuto a una Sezione del Corso	15
RF10	Il sistema deve offrire al Docente una funzionalità per rendere visibile o mantenere non pubblicato un Contenuto, attivandolo successivamente	16
RF11	Il sistema deve offrire al Docente una funzionalità per modificare un Contenuto pubblicato	17
RF12	Il sistema deve offrire al Docente una funzionalità per rimuovere un Contenuto pubblicato	18
RF13	Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Corsi a cui è iscritto	19
RF14	Il sistema deve offrire allo Studente una funzionalità per visualizzare l'elenco dei Contenuti di un Corso	20
RF15	Il sistema deve offrire allo Studente una funzionalità per filtrare i Contenuti di un Corso per categoria, data o Sezione	21
RF16	Il sistema deve offrire allo Studente una funzionalità per visualizzare il dettaglio di un Contenuto	22
RF17	Il sistema deve offrire al Docente una funzionalità per visualizzare l'elenco dei Contenuti pubblicati di un Corso	23
RF18	Il sistema deve offrire al Docente una funzionalità per visualizzare la distribuzione dei Contenuti per Categoria in un Corso	24
RF19	Il sistema deve offrire al Docente una funzionalità per monitorare l'andamento di un Corso (n. Contenuti pubblicati, n. Studenti iscritti, Categorie più utilizzate)	25
RF20	Il sistema deve offrire all'Utente una funzionalità per cercare Contenuti per titolo, descrizione o categoria	26
RF21	Il sistema deve offrire all'Utente una funzionalità per visualizzare il profilo di un Docente, comprensivo di nome ed elenco dei Corsi gestiti	27
RF22	Il sistema deve inviare una Notifica allo Studente quando viene pubblicato un nuovo Contenuto in un Corso a cui è iscritto	28
RF23	Il sistema deve inviare una Notifica allo Studente quando il Docente pubblica un avviso nel Corso	29

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RD01	Di ogni Utente si vuole memorizzare nome, cognome, email istituzionale, ruolo e credenziali di accesso	3
RD02	Di ogni Corso si vuole memorizzare titolo, descrizione, codice univoco, anno accademico (opzionale) e Docente titolare	6
RD03	Di ogni Iscrizione si vuole memorizzare lo Studente e il Corso associati	9
RD04	Di ogni Contenuto si vuole memorizzare titolo, descrizione, categoria, data di pubblicazione e stato di pubblicazione	11
RD05	Ogni Contenuto deve appartenere a una sola Categoria (slide, dispense, esercizi, soluzioni, avvisi, materiale integrativo)	12
RD06	Di ogni Sezione si vuole memorizzare titolo e Corso di appartenenza	14

2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010)

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUA L-01	Usabilità	L'interfaccia deve essere intuitiva e utilizzabile senza formazione specifica	Tempo di completamento di un'operazione base (es. pubblicazione materiale)	≤ 3 minuti	Test con utenti
RQ-QUA L-02	Sicurezza	Il sistema deve proteggere i dati personali e gestire correttamente i permessi tra Docenti e Studenti	N. accessi non autorizzati a risorse riservate	0	Test di controllo permessi
RQ-QUA L-03	Affidabilità	Il sistema deve essere disponibile agli utenti registrati	Uptime mensile	$\geq 99\%$	Monitoraggio del sistema

RQ- QUA L-04	Portabilità	La piattaforma deve essere accessibile da diverse piattaforme desktop (Windows, Linux, macOS) grazie alla portabilità della JVM su cui si basa Java Swing	Corretta visualizzazione su sistemi operativi/risoluzioni testate	100% piattaforme target	Test cross-platform (Windows/Linux/macOS)
RQ- QUA L-05	Efficienza prestazionale	Il sistema deve mostrare rapidamente l'elenco dei materiali richiesti	Tempo di risposta	≤ 2 secondi (95% richieste)	Test di carico

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
Vo1	Il sistema deve essere sviluppato in linguaggio Java, rispettando il pattern architetturale BCED (Boundary-Control-Entity-Database)	
Vo2	Il livello di persistenza deve essere realizzato mediante il pattern DAO, utilizzando il DBMS H2	
Vo3	L'interfaccia grafica deve essere realizzata mediante la libreria Java Swing	
Vo4	Per l'invio delle notifiche agli Studenti deve essere disponibile un servizio di messaggistica interno alla piattaforma	

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Docente
- Studente

Casi d'uso:

- **UC1:** Registrazione
- **UC2:** Accesso
- **UC3:** CreaCorso
- **UC4:** ModificaCorso
- **UC5:** IscrivitiStudente
- **UC6:** IscrivitiCorso
- **UC7:** PubblicaContenuto
- **UC8:** OrganizzaSezioni
- **UC9:** AttivaContenuto
- **UC10:** ModificaContenuto
- **UC11:** RimuoviContenuto
- **UC12:** VisualizzaCorsiIscritti
- **UC13:** VisualizzaContenutiCorso
- **UC14:** VisualizzaDettaglioContenuto
- **UC15:** MonitoraAndamentoCorso
- **UC16:** CercaContenuti
- **UC17:** VisualizzaProfiloDocente

Casi d'uso di inclusione:

- **UC18:** FiltraContenuti
- **UC19:** VisualizzaContenutiPubblicati
- **UC20:** VisualizzaDistribuzioneCategoria
- **UC21:** InviaNotifica

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1: Registrazione	Utente	-	-	RF01
UC2: Accesso	Utente	-	-	RF02
UC3: CreaCorso	Docente	-	-	RF03
UC4: ModificaCorso	Docente	-	-	RF04
UC5: IscrivitiStudente	Docente	-	-	RF05
UC6: IscrivitiCorso	Studente	-	-	RF06
UC7: PubblicaContenuto	Docente	-	<<include>> UC21	RF07
UC8: OrganizzaSezioni	Docente	-	-	RF08, RF09
UC9: AttivaContenuto	Docente	-	-	RF10
UC10: ModificaContenuto	Docente	-	-	RF11
UC11: RimuoviContenuto	Docente	-	-	RF12
UC12: VisualizzaCorsiIscritti	Studente	-	-	RF13
UC13: VisualizzaContenutiCorso	Studente	-	<<include>> UC18	RF14
UC14: VisualizzaDettaglioContenuto	Studente	-	-	RF16
UC15: MonitoraAndamentoCorso	Docente	-	<<include>> UC19, UC20	RF19
UC16: CercaContenuti	Utente	-	-	RF20

UC17: VisualizzaProfiloDocente	Utente	-	-	RF21
UC18: FiltraContenuti	Studente	-	<<included by>> UC13	RF15
UC19: VisualizzaContenutiPubblicati	Docente	-	<<included by>> UC15	RF17
UC20: VisualizzaDistribuzioneCategor ia	Docente	-	<<included by>> UC15	RF18
UC21: InviaNotifica	-	Studente	<<included by>> UC7	RF22, RF23

2.5.2 Diagramma dei casi d'uso

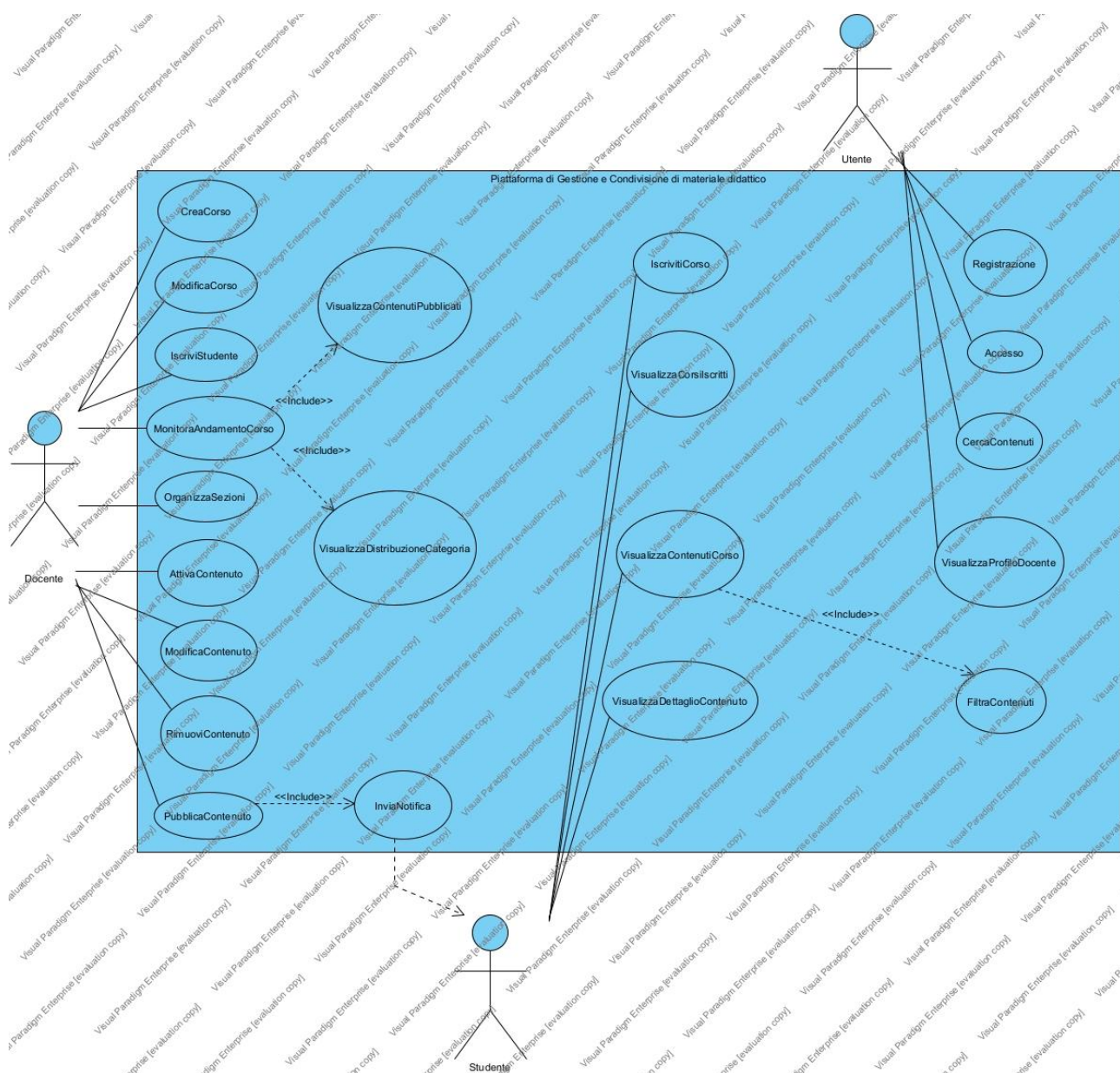


Diagramma dei casi d'uso, generato in Visual Paradigm.

2.5.3 Scenari

Si riporta di seguito lo scenario principale per il caso d'uso scelto per lo sviluppo di dettaglio: IscrivitiCorso.

Caso d'uso:	IscrivitiCorso (UC6)
Attore primario	Studente
Attore secondario	-
Descrizione	Uno Studente si iscrive autonomamente a un Corso inserendo il codice univoco fornito dal Docente.
Pre-Condizioni	Lo Studente ha effettuato l'accesso alla piattaforma.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente seleziona la funzionalità "Iscriviti a un corso" 2. Il sistema richiede allo Studente di inserire il codice univoco del Corso 3. Lo Studente inserisce il codice del Corso 4. Il sistema verifica che il codice corrisponda a un Corso esistente 5. if ci sono un Corso con quel codice 5.1. Il sistema verifica che lo Studente non sia già iscritto al Corso 5.2. if lo Studente non è già iscritto al Corso 5.2.1. Il sistema crea una nuova Iscrizione tra lo Studente e il Corso 5.2.2. Il sistema aggiunge il Corso all'elenco dei corsi seguiti dallo Studente 5.2.3. Il sistema conferma allo Studente l'avvenuta iscrizione
Post-Condizioni	Lo Studente risulta iscritto al Corso e il Corso compare nell'elenco dei corsi seguiti dallo Studente.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 4, se il codice non corrisponde a nessun Corso esistente, il sistema restituisce un messaggio di errore e richiede allo Studente di reinserire il codice (torna al punto 2) Al punto 5.1, se lo Studente risulta già iscritto al Corso, il sistema restituisce un messaggio informativo e termina il caso d'uso

PubblicaContenuto (UC7)

Scenario a cura del membro del gruppo responsabile del caso d'uso PubblicaContenuto.

Caso d'uso:	PubblicaContenuto (UC7)
Attore primario	Docente
Attore secondario	-
Descrizione	Il Docente pubblica un nuovo Contenuto didattico all'interno di un Corso, specificandone titolo, descrizione, categoria e sezione di appartenenza.
Pre-Condizioni	Il Docente ha effettuato l'accesso ed è titolare del Corso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente seleziona la funzionalità "Pubblica materiale" all'interno di un proprio Corso 2. Il sistema richiede al Docente titolo, descrizione, categoria, sezione (opzionale) e stato di pubblicazione del Contenuto 3. Il Docente inserisce i dati richiesti 4. Il sistema verifica che i dati obbligatori siano stati correttamente specificati 5. if i dati sono validi 5.1. Il sistema crea il nuovo Contenuto e lo associa al Corso (e alla Sezione, se indicata) 5.2. if il Contenuto è impostato come visibile immediatamente 5.2.1. «include» InviaNotifica 5.3. Il sistema conferma al Docente l'avvenuta pubblicazione
Post-Condizioni	Il nuovo Contenuto è stato creato e associato al Corso (e alla Sezione, se indicata); se reso visibile immediatamente, gli Studenti iscritti al Corso sono stati notificati.
Casi d'uso correlati	InviaNotifica

Sequenza di eventi alternativi	Al punto 4, se i dati obbligatori non sono stati specificati correttamente, il sistema restituisce un messaggio di errore e richiede al Docente di correggerli (torna al punto 3)
---------------------------------------	---

MonitoraAndamentoCorso (UC15)

Scenario a cura del membro del gruppo responsabile del caso d'uso MonitoraAndamentoCorso.

Caso d'uso:	MonitoraAndamentoCorso (UC15)
Attore primario	Docente
Attore secondario	-
Descrizione	Il Docente consulta le statistiche di andamento di un proprio Corso: numero di materiali pubblicati, numero di Studenti iscritti e categorie di Contenuti più utilizzate.
Pre-Condizioni	Il Docente ha effettuato l'accesso ed è titolare del Corso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Docente seleziona un proprio Corso e richiede la funzionalità "Andamento corso" 2. «include» VisualizzaContenutiPubblicati 3. Il sistema calcola il numero totale di Contenuti pubblicati nel Corso 4. Il sistema calcola il numero di Studenti iscritti al Corso 5. «include» VisualizzaDistribuzioneCategoria 6. Il sistema calcola la distribuzione dei Contenuti per Categoria 7. Il sistema mostra al Docente il riepilogo dell'andamento del Corso
Post-Condizioni	Il Docente ha visualizzato il riepilogo aggiornato dell'andamento del Corso (numero di Contenuti pubblicati, numero di Studenti iscritti, distribuzione per Categoria).
Casi d'uso correlati	VisualizzaContenutiPubblicati, VisualizzaDistribuzioneCategoria
Sequenza di eventi alternativi	Ai punti 3-6, se il Corso non ha ancora Contenuti pubblicati né Studenti iscritti, i valori calcolati risultano pari a zero; il sistema mostra comunque il riepilogo al punto 7

VisualizzaContenutiCorso (UC13)

Scenario a cura del membro del gruppo responsabile del caso d'uso VisualizzaContenutiCorso.

Caso d'uso:	VisualizzaContenutiCorso (UC13)
Attore primario	Studente
Attore secondario	-
Descrizione	Lo Studente consulta l'elenco dei materiali didattici pubblicati in un Corso a cui è iscritto, eventualmente filtrandoli per categoria, data o sezione.
Pre-Condizioni	Lo Studente ha effettuato l'accesso ed è iscritto al Corso.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente seleziona un Corso a cui è iscritto 2. Il sistema mostra l'elenco dei Contenuti pubblicati nel Corso 3. Lo Studente può impostare uno o più filtri (categoria, data, sezione) 4. if lo Studente ha impostato dei filtri 4.1. «include» FiltraContenuti 4.2. Il sistema mostra l'elenco dei Contenuti filtrato 5. Lo Studente seleziona un Contenuto per consultarne i dettagli 6. Il sistema mostra titolo, descrizione, categoria e data di pubblicazione del Contenuto selezionato
Post-Condizioni	Lo Studente ha visualizzato l'elenco (eventualmente filtrato) dei Contenuti del Corso e, se richiesto, il dettaglio di un Contenuto selezionato.
Casi d'uso correlati	FiltraContenuti
Sequenza di eventi alternativi	Al punto 4.1, se nessun Contenuto soddisfa i filtri impostati, il sistema mostra un messaggio informativo e un elenco vuoto

2.6 Diagramma delle classi

Il diagramma delle classi di analisi (System Domain Model) è stato modellato tramite classi Java (package dominio, cartella AnalysisModel/) da importare in Visual Paradigm con la funzione Create UML → Reverse Java Code.

Le classi del modello di dominio sono:

- Utente (astratta)
- Docente (estende Utente)
- Studente (estende Utente)
- Corso
- Sezione
- Contenuto
- Categoria (enumerazione)
- Iscrizione (classe associativa Studente-Corso)
- Notifica
- Piattaforma (Information Expert dell'elenco Utenti)

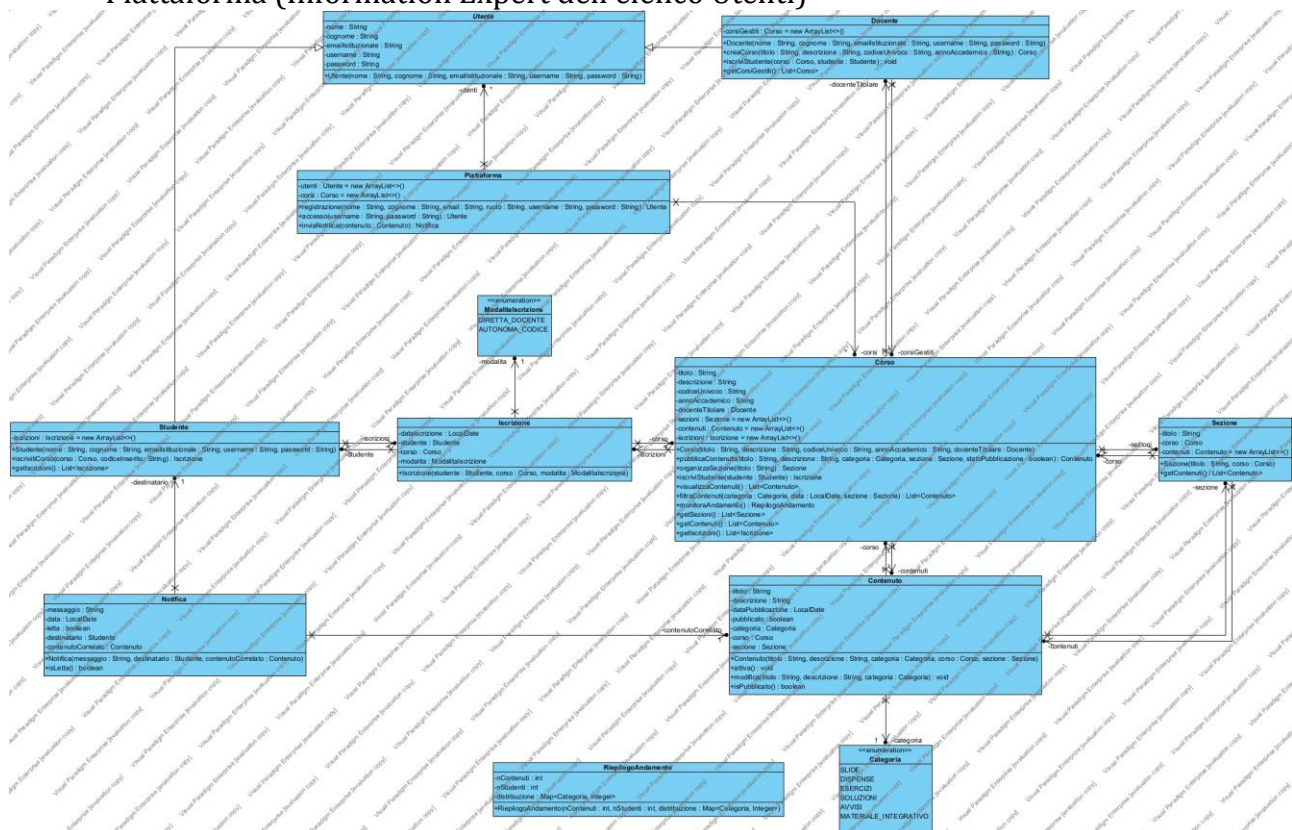


Diagramma delle classi di analisi, generato in Visual Paradigm tramite reverse engineering del package dominio.

RESPONSABILITÀ	CLASSE
Registrazione	Piattaforma
Accesso	Piattaforma
CreaCorso	Docente
IscrivitiCorso	Studente
PubblicaContenuto	Corso
OrganizzaSezioni	Corso
VisualizzaContenutiCorso	Corso
MonitoraAndamentoCorso	Corso
InviaNotifica	Piattaforma

Registrazione e Accesso sono responsabilità di **Piattaforma**, in quanto <<information expert>> dell'elenco degli Utenti registrati.

CreaCorso è responsabilità di **Docente**, perché è <<Creator>> della classe Corso (il Docente la possiede e la gestisce).

IscrivitiCorso è responsabilità di **Studente**, perché è <<Creator>> della classe Iscrizione.

PubblicaContenuto e *OrganizzaSezioni* sono responsabilità di **Corso**, in quanto <<Creator>> rispettivamente di Contenuto e di Sezione, ed <<information expert>> della propria composizione.

VisualizzaContenutiCorso e *MonitoraAndamentoCorso* sono responsabilità di **Corso**, in quanto <<information expert>> dei propri Contenuti e delle proprie Iscrizioni.

InviaNotifica è responsabilità di **Piattaforma**, in quanto coordina l'invio delle Notifiche agli Studenti iscritti ai Corsi interessati.

2.7 Diagrammi di sequenza

Si riportano di seguito i diagrammi di sequenza di analisi per i 4 casi d'uso scelti dai membri del gruppo per lo sviluppo di dettaglio, costruiti a partire dagli scenari descritti in 2.5.3. In accordo con la modellazione a livello di analisi, il sistema è rappresentato mediante un unico oggetto generico (Piattaforma); compaiono ulteriori oggetti del dominio solo in corrispondenza della creazione di una nuova istanza («create»), come indicato a lezione.

IscrivitiCorso (UC6)

Diagramma di sequenza di analisi a cura del membro del gruppo responsabile del caso d'uso IscrivitiCorso.

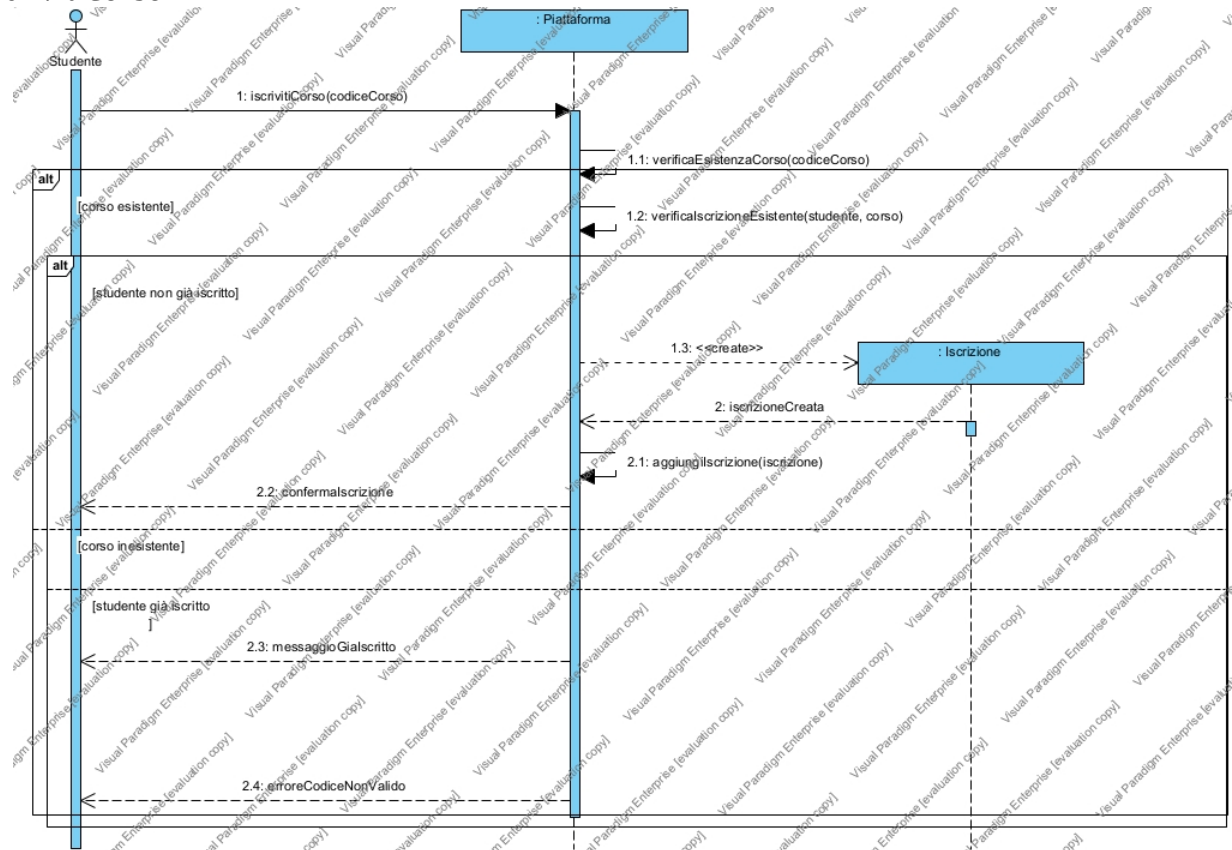


Diagramma di sequenza di analisi di IscrivitiCorso, generato in Visual Paradigm.

PubblicaContenuto (UC7)

Diagramma di sequenza di analisi a cura del membro del gruppo responsabile del caso d'uso PubblicaContenuto.

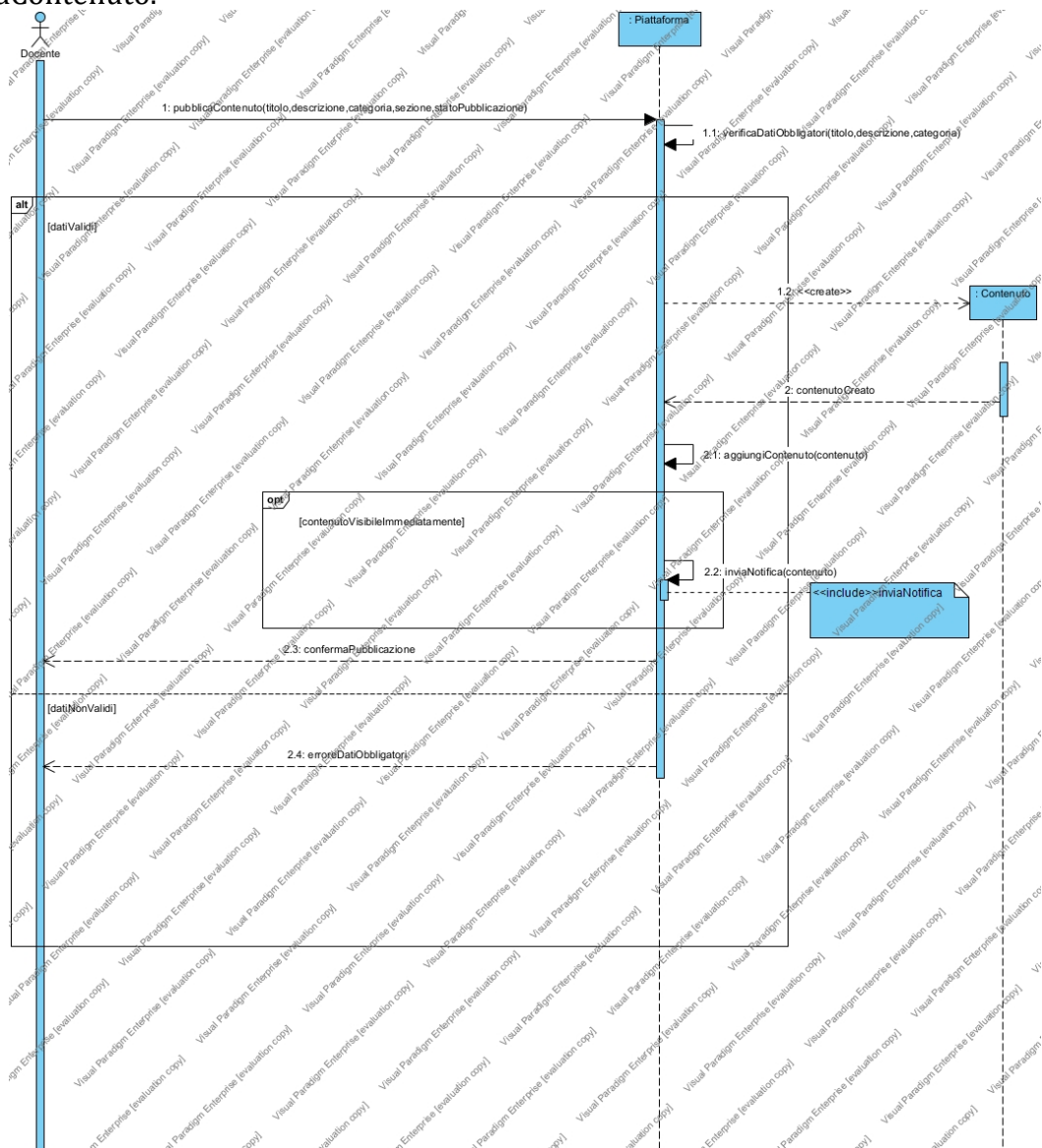


Diagramma di sequenza di analisi di PubblicaContenuto, generato in Visual Paradigm.

MonitoraAndamentoCorso (UC15)

Diagramma di sequenza di analisi a cura del membro del gruppo responsabile del caso d'uso MonitoraAndamentoCorso.

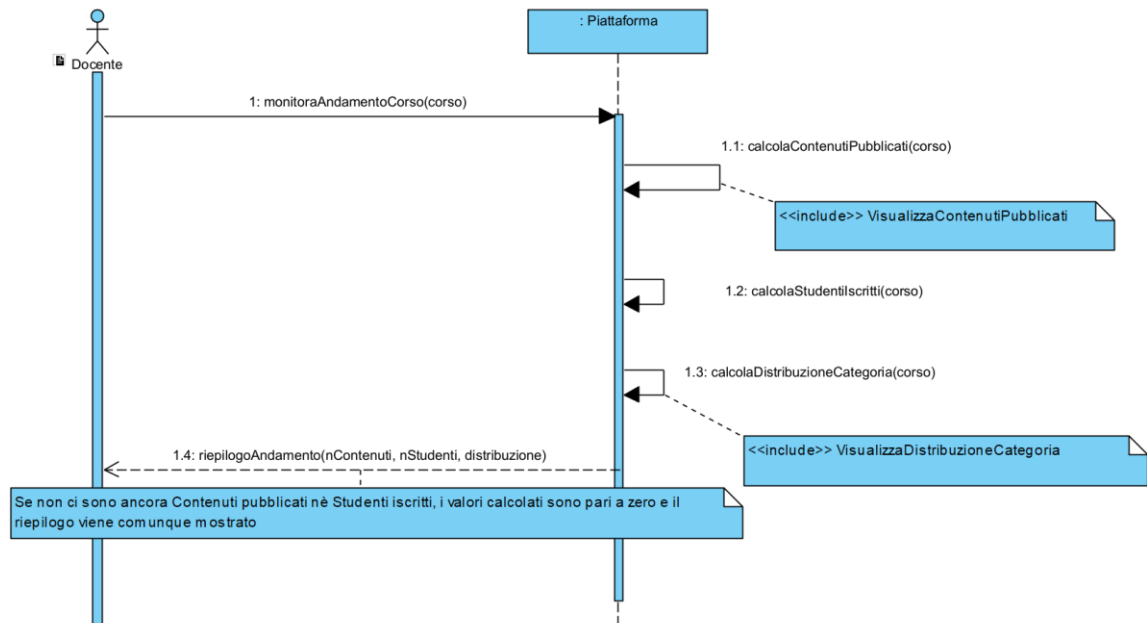


Diagramma di sequenza di analisi di MonitoraAndamentoCorso, generato in Visual Paradigm.

VisualizzaContenutiCorso (UC13)

Diagramma di sequenza di analisi a cura del membro del gruppo responsabile del caso d'uso VisualizzaContenutiCorso.

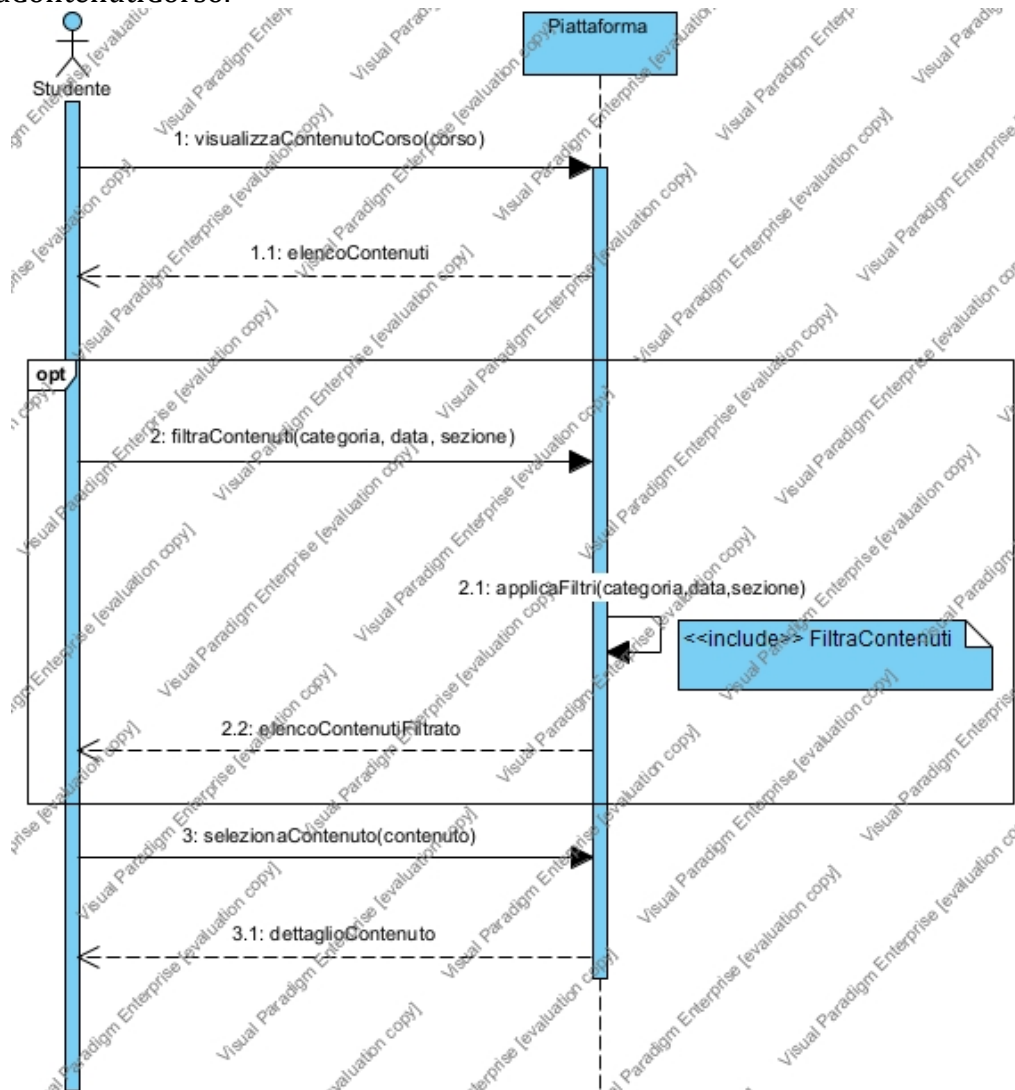


Diagramma di sequenza di analisi di VisualizzaContenutiCorso, generato in Visual Paradigm.

3. Piano di test funzionale

Si intende progettare i casi di test funzionale con la tecnica del Category Partition Testing per i 4 casi d'uso scelti dai membri del gruppo per lo sviluppo di dettaglio: IscrivitiCorso, PubblicaContenuto, MonitoraAndamentoCorso, VisualizzaContenutiCorso.

3.1 IscrivitiCorso

ISCRIVITICORSO		
CODICE CORSO	CORSO CORRISPONDENTE	STATO ISCRIZIONE STUDENTE
- Stringa alfanumerica di lunghezza valida (8-12 caratteri) - Stringa vuota [ERROR] - Stringa contenente caratteri non alfanumerici [ERROR]	- Il codice corrisponde a un Corso esistente - Il codice non corrisponde a nessun Corso esistente [ERROR]	- Lo Studente non è ancora iscritto al Corso - Lo Studente è già iscritto al Corso [ERROR]

Stringa di lunghezza eccessiva (> 12 caratteri) [ERROR]		
---	--	--

Il numero di test da effettuarsi senza particolari vincoli è: $4 * 2 * 2 = 16$.
 Con i vincoli **[ERROR]**, il numero di test da eseguire per testare singolarmente i vincoli è 5 (3 per Codice Corso, 1 per Corso Corrispondente, 1 per Stato Iscrizione Studente).
 Il numero di test risultante è: $(1*1*1) + 5 = 6$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Codice valido Corso corrispondente Studente non iscritto	Lo Studente ha effettuato l'accesso	{codiceCorso: "CINF2024A"}	Iscrizione creata; messaggio di conferma avvenuta iscrizione	Lo Studente risulta iscritto al Corso
2	Codice corso vuoto	Codice vuoto [ERROR] restanti validi	Lo Studente ha effettuato l'accesso	{codiceCorso: ""}	Messaggio di errore: codice non valido	Nessuna Iscrizione viene creata
3	Codice corso con simboli	Codice con simboli [ERROR] restanti validi	Lo Studente ha effettuato l'accesso	{codiceCorso: "CI@NF#"}	Messaggio di errore: codice non valido	Nessuna Iscrizione viene creata
4	Codice corso troppo lungo	Codice lunghezza eccessiva [ERROR] restanti validi	Lo Studente ha effettuato l'accesso	{codiceCorso: "CINF2024A0000000"}	Messaggio di errore: codice non valido	Nessuna Iscrizione viene creata
5	Codice non corrispondente a nessun Corso	Corso corrispondente [ERROR] restanti validi	Lo Studente ha effettuato l'accesso	{codiceCorso: "ZZZZ9999"}	Messaggio di errore: nessun Corso trovato con questo codice	Nessuna Iscrizione viene creata
6	Studente già iscritto al Corso	Stato iscrizione [ERROR] restanti validi	Lo Studente ha effettuato l'accesso ed è già iscritto al Corso	{codiceCorso: "CINF2024A"}	Messaggio informativo: Studente già iscritto al Corso	Lo stato del sistema resta invariato

3.2 PubblicaContenuto

PUBBLICACONTENUTO				
TITOLO	DESCRIZIONE	CATEGORIA	SEZIONE (opzionale)	STATO PUBBLICAZIONE
• Stringa di lunghezza <= 100 caratteri • Stringa vuota [ERROR] • Stringa di lunghezza > 100 caratteri [ERROR]	• Stringa di lunghezza <= 500 caratteri • Stringa vuota [ERROR] • Stringa di lunghezza > 500 caratteri [ERROR]	• Valore appartenente all'enumerazione Categoria • Valore nullo/non specificato [ERROR] • Valore non appartenente all'enumerazione Categoria [ERROR]	• Nessuna Sezione specificata • Sezione esistente e appartenente al Corso • Sezione non appartenente al Corso [ERROR]	• Valore true (visibile immediatamente) • Valore false (non pubblicato)

Il numero di test da effettuarsi senza particolari vincoli è: $3 * 3 * 3 * 3 * 2 = 162$.

Le categorie Sezione e Stato Pubblicazione presentano più di una classe valida: sfruttando la possibilità di combinare più valori validi nello stesso caso di test (weak equivalence class testing), sono sufficienti 2 casi di test, anziché $1*1*1*2*2 = 4$, per coprire tutte le classi valide di tutte le categorie.

Con i vincoli [ERROR], il numero di test da eseguire per testare singolarmente i vincoli è 7 (2 per Titolo, 2 per Descrizione, 2 per Categoria, 1 per Sezione).

Il numero di test risultante è: $2 + 7 = 9$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi (pubblicazione immediata, senza Sezione)	Titolo valido Descrizione valida Categoria valida Sezione: nessuna Stato pubblicazione: true	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Slide Lezione 1", descrizione: "Introduzione al corso", categoria: SLIDE, sezione: null, statoPubblicazione: true}	Contenuto creato e pubblicato; Studenti iscritti notificati	Il Contenuto risulta pubblicato nel Corso
2	Tutti input validi (non pubblicato, con Sezione)	Titolo valido Descrizione valida Categoria valida Sezione: esistente Stato pubblicazione: false	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Esercizi Modulo 2", descrizione: "Esercitazione", categoria: ESERCIZI, sezione: "Modulo 2", statoPubblicazione: false}	Contenuto creato ma non visibile agli Studenti	Il Contenuto risulta creato ma non pubblicato
3	Titolo vuoto	Titolo [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "", descrizione: "Introduzione", categoria: SLIDE, sezione: null, statoPubblicazione: true}	Messaggio di errore: dati obbligatori non specificati	Nessun Contenuto viene creato
4	Titolo troppo lungo	Titolo [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Titolo eccessivamente lungo (>100 caratteri)...", descrizione: "Introduzione", categoria: SLIDE, sezione: null, statoPubblicazione: true}	Messaggio di errore: dati obbligatori non specificati	Nessun Contenuto viene creato
5	Descrizione vuota	Descrizione [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Slide Lezione 1", descrizione: "", categoria: SLIDE, sezione: null, statoPubblicazione: true}	Messaggio di errore: dati obbligatori non specificati	Nessun Contenuto viene creato
6	Descrizione troppo lunga	Descrizione [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Slide Lezione 1", descrizione: "Descrizione eccessivamente lunga (>500 caratteri)...", categoria: SLIDE, sezione: null, statoPubblicazione: true}	Messaggio di errore: dati obbligatori non specificati	Nessun Contenuto viene creato
7	Categoria non specificata	Categoria [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Slide Lezione 1", descrizione: "Introduzione", categoria: null, sezione: null, statoPubblicazione: true}	Messaggio di errore: dati obbligatori non specificati	Nessun Contenuto viene creato
8	Categoria non valida	Categoria [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Slide Lezione 1", descrizione: "Introduzione", categoria: "VIDEO", sezione: null, statoPubblicazione: true}	Messaggio di errore: categoria non valida	Nessun Contenuto viene creato

			è titolare del Corso			
9	Sezione non appartenente al Corso	Sezione [ERROR] restanti validi	Il Docente ha effettuato l'accesso ed è titolare del Corso	{titolo: "Slide Lezione 1", descrizione: "Introduzione", categoria: SLIDE, sezione: "Sezione di un altro Corso", statoPubblicazione: true}	Messaggio di errore: Sezione non valida per questo Corso	Nessun Contenuto viene creato

3.3 MonitoraAndamentoCorso

MONITORAANDAMENTOCORSO		
STATO CONTENUTI DEL CORSO	STATO ISCRIZIONI DEL CORSO	TITOLARITÀ DEL CORSO
- Il Corso ha almeno un Contenuto pubblicato - Il Corso non ha Contenuti pubblicati	- Il Corso ha almeno uno Studente iscritto - Il Corso non ha Studenti iscritti	- Il Docente richiedente è titolare del Corso - Il Docente richiedente non è titolare del Corso [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $2 * 2 * 2 = 8$.

Le categorie Stato Contenuti del Corso e Stato Iscrizioni del Corso presentano entrambe due classi valide: combinandole a coppie (weak equivalence class testing) sono sufficienti 2 casi di test, anziché $2*2 = 4$, per coprire tutte le classi valide di queste due categorie.

Con i vincoli [ERROR], il numero di test da eseguire per testare singolarmente i vincoli è 1 (Titolarità del Corso).

Il numero di test risultante è: $2 + 1 = 3$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Corso con Contenuti pubblicati e Studenti iscritti	Contenuti presenti Iscrizioni presenti Titolarità valida	Il Docente ha effettuato l'accesso ed è titolare del Corso	{corso: "Ingegneria del Software"} (5 Contenuti pubblicati, 30 Studenti iscritti)	Riepilogo con nContenuti=5 , nStudenti=30 , distribuzione per Categoria	Il Docente visualizza il riepilogo aggiornato
2	Corso senza Contenuti pubblicati e senza Studenti iscritti	Contenuti assenti Iscrizioni assenti Titolarità valida	Il Docente ha effettuato l'accesso ed è titolare del Corso	{corso: "Corso appena creato"} (0 Contenuti, 0 Studenti)	Riepilogo con nContenuti=0 , nStudenti=0, distribuzione vuota	Il Docente visualizza comunque il riepilogo
3	Docente non titolare del Corso	Titolarità [ERROR]	Il Docente ha effettuato l'accesso ma non è titolare del Corso	{corso: "Corso di un altro Docente"}	Messaggio di errore: accesso non autorizzato	Il riepilogo non viene mostrato

3.4 VisualizzaContenutiCorso

VISUALIZZACONTENUTICORSO		
ISCRIZIONE DELLO STUDENTE AL CORSO	FILTRO IMPOSTATO	CONTENUTO SELEZIONATO PER IL DETTAGLIO

- Lo Studente è iscritto al Corso richiesto - Lo Studente non è iscritto al Corso richiesto [ERROR]	- Nessun filtro impostato (visualizza tutti i Contenuti) - Filtro impostato con almeno un Contenuto corrispondente - Filtro impostato senza Contenuti corrispondenti	- Contenuto esistente e appartenente al Corso - Contenuto inesistente o non appartenente al Corso [ERROR]
---	--	---

Il numero di test da effettuarsi senza particolari vincoli è: $2 * 3 * 2 = 12$.

La categoria Filtro Impostato presenta tre classi valide; le altre due categorie ne presentano una sola: sono quindi necessari 3 casi di test per coprire tutte le classi valide di Filtro Impostato, combinate con l'unica classe valida delle altre due categorie.

Con i vincoli [ERROR], il numero di test da eseguire per testare singolarmente i vincoli è 2 (1 per Iscrizione dello Studente, 1 per Contenuto Selezionato).

Il numero di test risultante è: $3 + 2 = 5$.

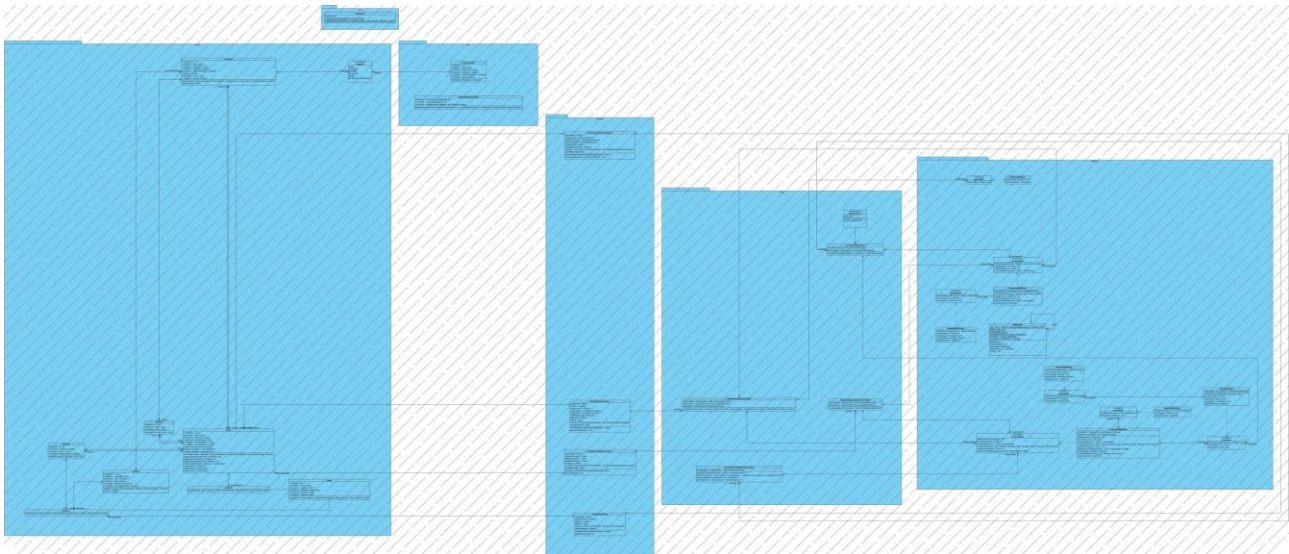
TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Nessun filtro impostato	Iscrizione valida Nessun filtro Contenuto valido	Lo Studente ha effettuato l'accesso ed è iscritto al Corso	{corso: "Ingegneria del Software", filtro: nessuno}	Elenco completo dei Contenuti pubblicati nel Corso	Lo Studente visualizza l'elenco completo
2	Filtro impostato con risultati	Iscrizione valida Filtro con risultati Contenuto valido	Lo Studente ha effettuato l'accesso ed è iscritto al Corso	{corso: "Ingegneria del Software", filtro: {categoria: SLIDE}}	Elenco filtrato non vuoto dei Contenuti di categoria SLIDE	Lo Studente visualizza l'elenco filtrato
3	Filtro impostato senza risultati	Iscrizione valida Filtro senza risultati Contenuto valido	Lo Studente ha effettuato l'accesso ed è iscritto al Corso	{corso: "Ingegneria del Software", filtro: {categoria: AVVISI}} (nessun avviso pubblicato)	Messaggio informativo ed elenco vuoto	Lo Studente visualizza un elenco vuoto
4	Studente non iscritto al Corso	Iscrizione [ERROR]	Lo Studente ha effettuato l'accesso ma non è iscritto al Corso richiesto	{corso: "Corso a cui non è iscritto"}	Messaggio di errore: accesso non autorizzato	L'elenco dei Contenuti non viene mostrato
5	Contenuto selezionato inesistente	Contenuto selezionato [ERROR]	Lo Studente ha effettuato l'accesso ed è iscritto al Corso	{contenuto: "identificativo non esistente o di un altro Corso"}	Messaggio di errore: Contenuto non trovato	Il dettaglio del Contenuto non viene mostrato

4. Progettazione

4.1 Diagramma delle classi

Si riporta di seguito il diagramma delle classi di progettazione, organizzato nei package Boundary, Controller, Entity e Database secondo il pattern architetturale BCED. Le scelte di traduzione dal modello di analisi (reificazione delle classi associative, verso di navigabilità delle

associazioni, specifica di argomenti e tipo di ritorno delle operazioni) sono descritte nel paragrafo 4.1.1.



4.1.1 Traduzione classi ed associazioni

Nel passaggio dal diagramma delle classi di analisi a quello di progettazione sono state operate le scelte seguenti, coerentemente con il pattern BCED adottato e con l'approccio Use Case Controller scelto per il livello Control.

Classe Piattaforma. nel modello di analisi rappresentava, per convenzione, l'unico oggetto di sistema a cui l'attore invia gli eventi (in coerenza con i diagrammi di sequenza di analisi, che mostrano un solo oggetto generico «: Piattaforma»). A livello di progettazione questo ruolo di coordinamento viene assunto dai quattro Use Case Controller (IscrivitiCorsoController, PubblicaContenutoController, MonitoraAndamentoCorsoController, VisualizzaContenutiCorsoController): la classe Piattaforma viene pertanto rimossa dal modello di progetto, e i metodi che vi erano definiti (registrazione, accesso, inviaNotifica) sono stati ridistribuiti al Controller del caso d'uso competente, oppure trasformati in operazioni di query dei DAO (vedi 4.1.2.4 Package Database).

Reificazione della classe associativa Iscrizione. l'associazione «si iscrive a» tra Studente e Corso presentava già nel modello di analisi un attributo proprio (dataIscrizione): per questo era stata reificata nella classe associativa Iscrizione. Tale scelta viene confermata anche a livello di progettazione.

Verso di navigabilità delle associazioni. per ridurre l'accoppiamento tra le Entity ed evitare di dover mantenere sincronizzate collezioni in memoria con il contenuto del database, alcune associazioni sono state rese unidirezionali: Corso conosce il proprio Docente titolare (Corso → Docente), ma Docente non mantiene più un elenco esplicito dei corsiGestiti; analogamente, Iscrizione conosce Studente e Corso, ma Studente non mantiene più un elenco esplicito di iscrizioni. Le interrogazioni che nel modello di analisi erano effettuate navigando queste associazioni (ad esempio “verifica se lo Studente è già iscritto al Corso”) sono ora delegate alle classi DAO corrispondenti (IscrizioneDAO.verificaEsistente, IscrizioneDAO.contaPerCorso, IscrizioneDAO.cercaIscrittiPerCorso), che interrogano direttamente il database H2: in questo modo il dato usato per la decisione è sempre quello aggiornato in persistenza, e non una copia in memoria potenzialmente disallineata.

Specifica di argomenti e tipo di ritorno. i metodi individuati nel modello di analisi sono stati riportati nelle classi Control (i quattro Use Case Controller) con argomenti e tipo di ritorno specificati: ad esempio `IscrivitiCorsoController.iscrivitiCorso(Studiante, String)` restituisce un valore dell'enumerazione `EsitoIscrizione` (`SUCCESSO`, `STUDENTE_GIA_ISCRITTO`, `CORSO_INESISTENTE`), che sostituisce a livello di progetto i tre rami alternativi già individuati nello scenario e nel diagramma di sequenza di analisi del caso d'uso.

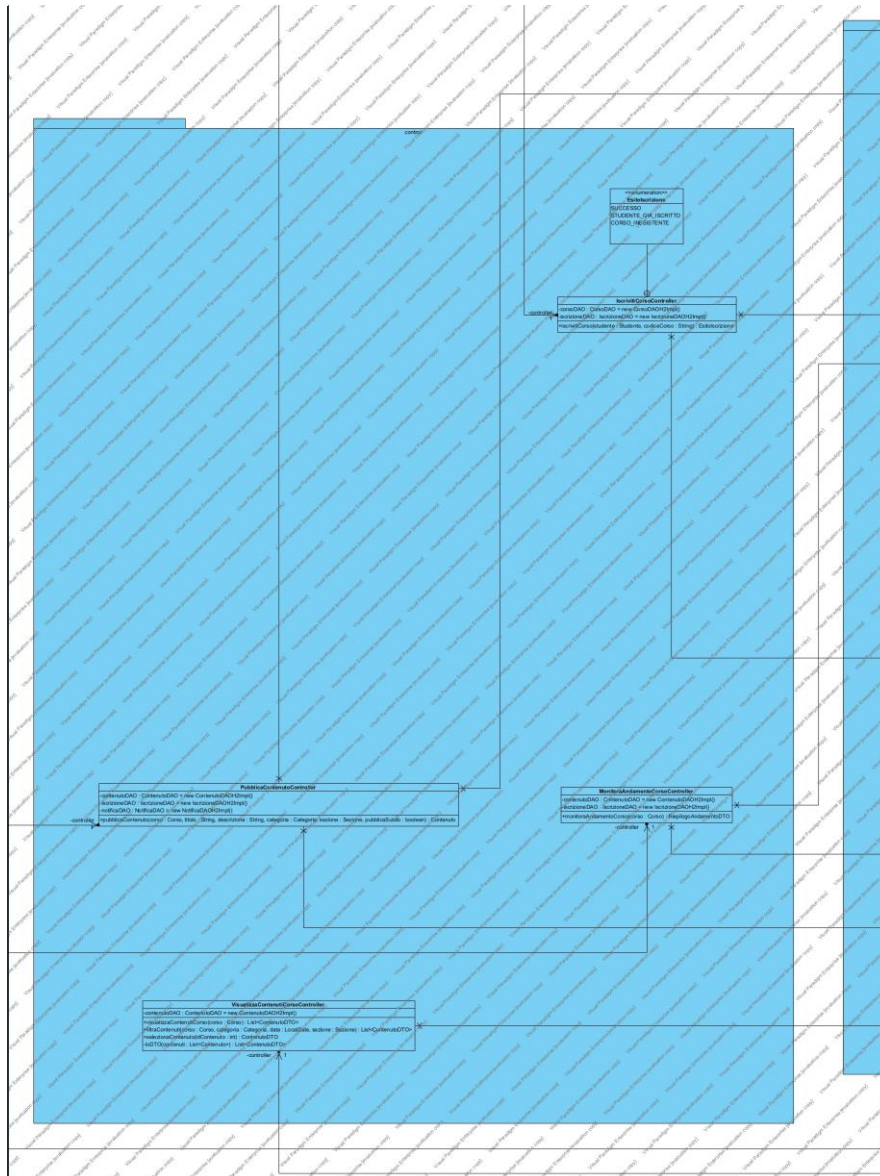
Spostamento della validazione di formato nella Boundary. seguendo l'indicazione delle slide del corso sul pattern BCED (“perché validare l'input nella GUI”), il controllo di presenza/formato dei campi obbligatori (ad esempio titolo, descrizione e categoria in `PubblicaContenuto`, o il solo formato del codiceCorso in `IscrivitiCorso`) è stato spostato dal Control alla Boundary, tramite la classe di utilità `Validazione` (package utility, estraneo al pattern BCED). Il Controller viene invocato dalla Boundary solo se questo controllo preliminare ha esito positivo, e si occupa esclusivamente delle regole di business che richiedono l'accesso al database (esistenza del Corso, Studiante già iscritto, ecc.).

Introduzione del package DTO. per evitare che le classi Boundary debbano conoscere la struttura interna delle Entity quando visualizzano collezioni (ad esempio l'elenco dei Contenuti in `VisualizzaContenutiCorso`, o il riepilogo numerico in `MonitoraAndamentoCorso`), sono stati introdotti gli oggetti `ContenutoDTO` e `RiepilogoAndamentoDTO` nel package `dto`, estraneo al pattern BCED, con lo stesso razionale descritto nel par. 5.5 del template.

4.1.2 Pattern BCED

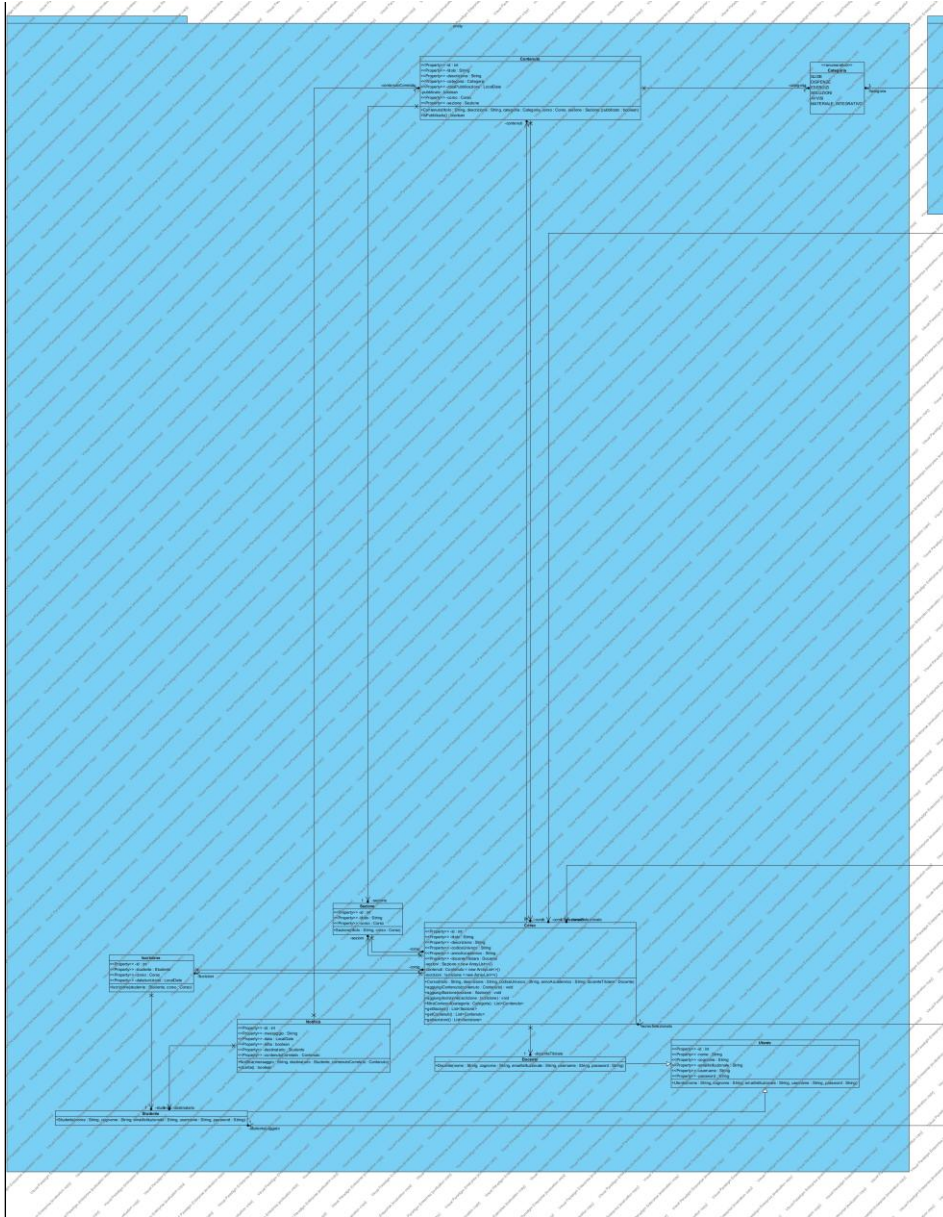
4.1.2.2 Package Controller

Questo package contiene gli oggetti che percepiscono gli eventi generati dalle interazioni con l'interfaccia utente e ne demandano la gestione all'unico componente del sistema software responsabile della gestione della Business Logic, il package Entity.



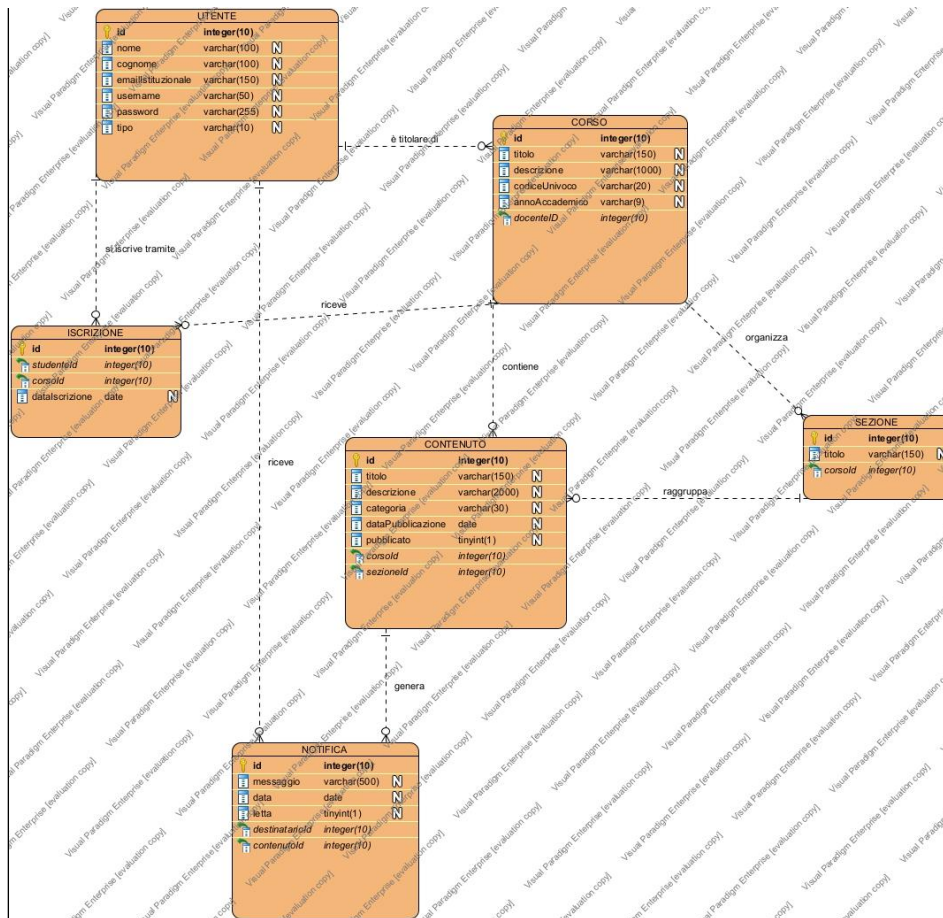
4.1.2.3 Package Entity

Il Package Entity contiene tutti gli oggetti che rappresentano la semantica delle entità del dominio applicativo e corrispondono alle strutture dati presenti all'interno del database di persistenza.



4.1.2.4 Package Database

Per il database di persistenza è stato adottato H2 in modalità embedded. Il modello Entity-Relationship, coerente con le scelte di traduzione classi-associazioni descritte al paragrafo 4.1.1 e con lo script di creazione delle tabelle (schema.sql), è il seguente:



Il Package Database contiene tutte le classi responsabili dell'estrazione dei dati dal DB, esponendo una vera e propria interfaccia che di fatto rende indipendenti le classi della Business Logic (Entity) dalla tecnologia di persistenza utilizzata.

In particolare, tra le strategie di risoluzione del problema dell'**impedance mismatch**, che nasce dalla mancata corrispondenza tra il modello Object Oriented e quello relazionale, si è deciso di adottare quella delle classi **DAO** (Data Access Objects), che consiste nell'utilizzo di appositi oggetti per l'accesso ai dati.

IscrivitiCorso (UC6)

Diagramma di sequenza di progetto a cura del membro del gruppo responsabile del caso d'uso IscrivitiCorso.

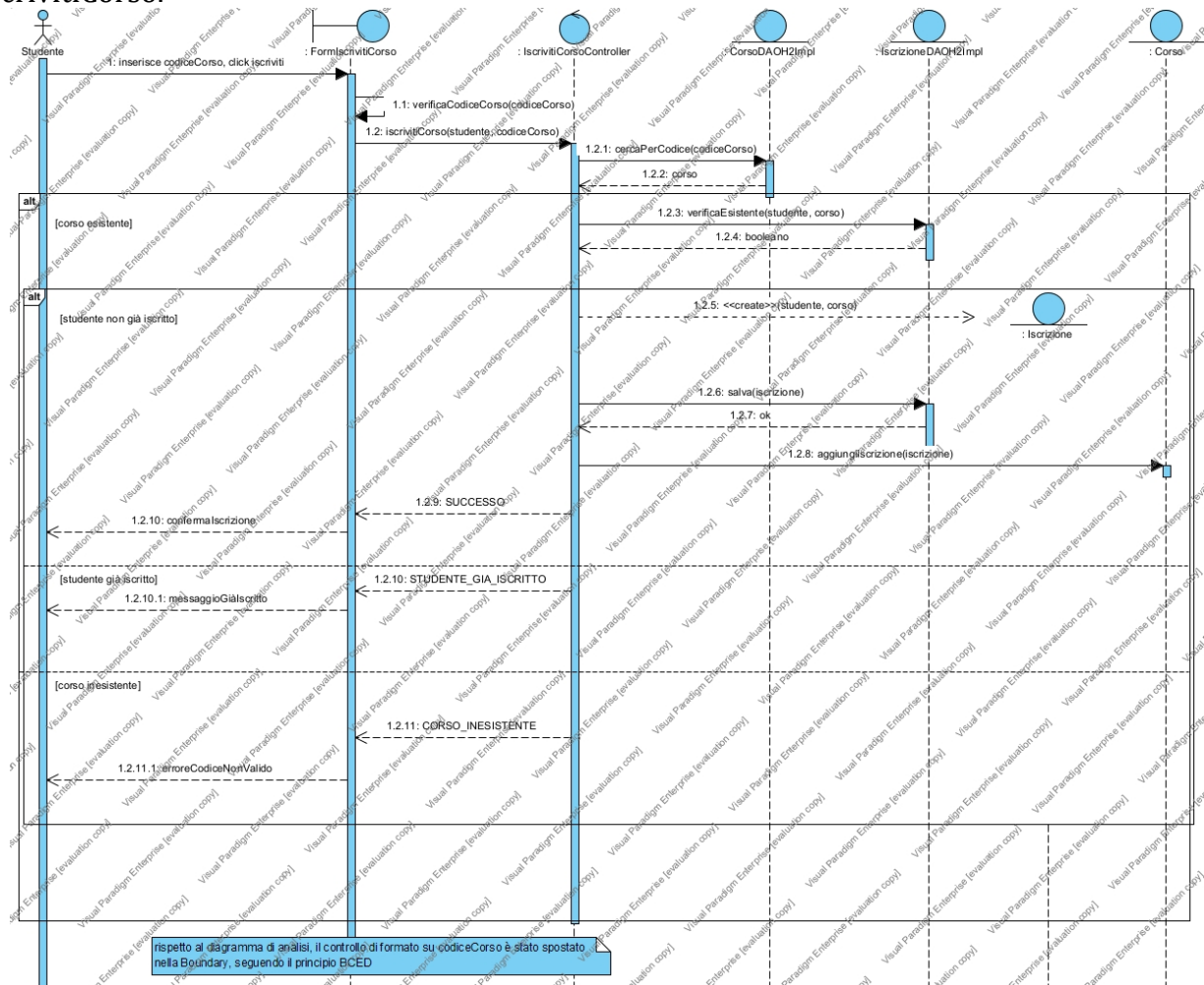
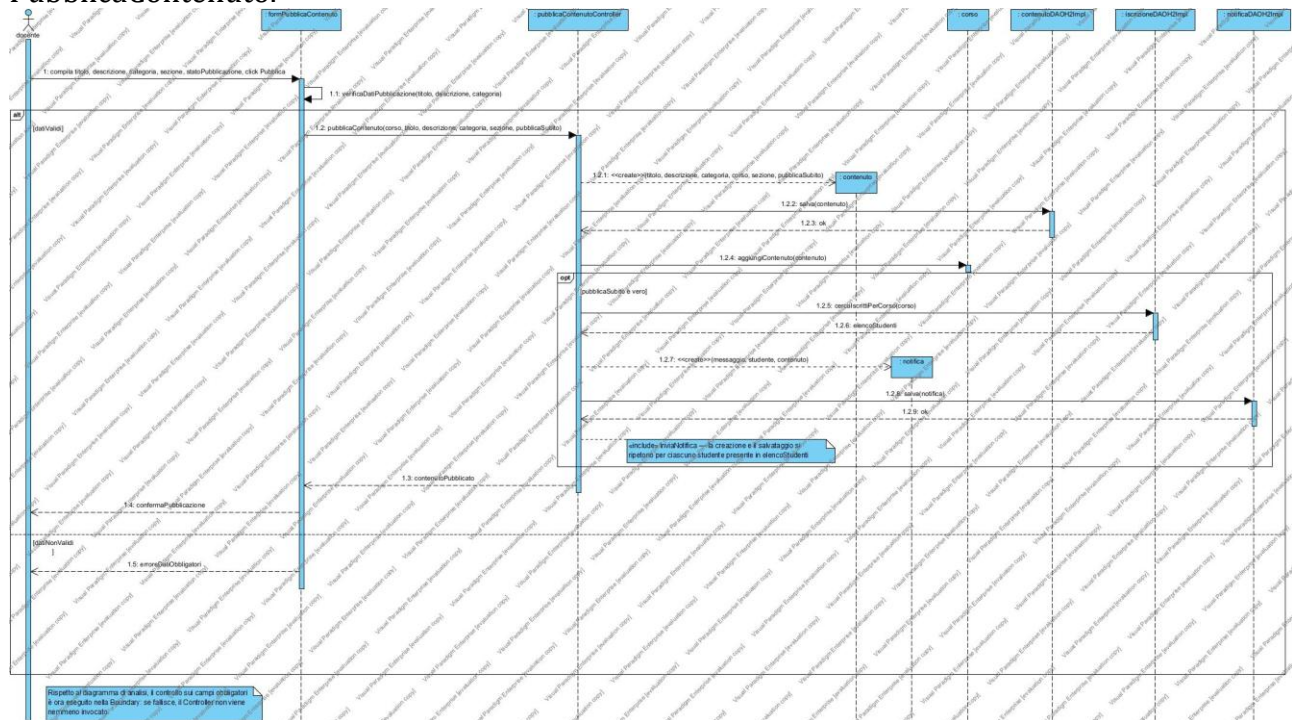


Diagramma di sequenza di progetto di IscrivitiCorso, generato in Visual Paradigm.

PubblicaContenuto (UC7)

Diagramma di sequenza di progetto a cura del membro del gruppo responsabile del caso d'uso PubblicaContenuto.



MonitoraAndamentoCorso (UC15)

Diagramma di sequenza di progetto a cura del membro del gruppo responsabile del caso d'uso MonitoraAndamentoCorso.

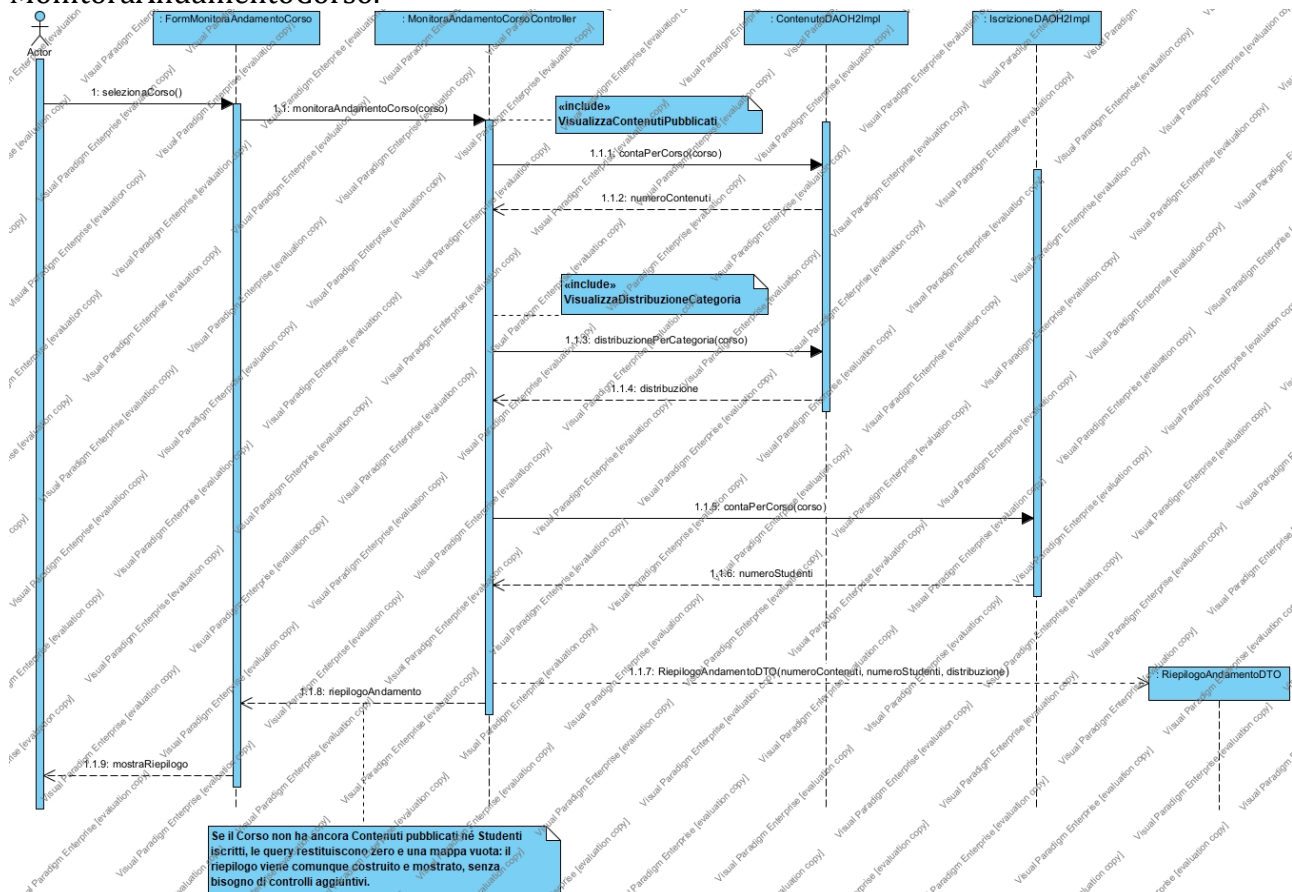


Diagramma di sequenza di progetto di MonitoraAndamentoCorso, generato in Visual Paradigm.

VisualizzaContenutiCorso (UC13)

Diagramma di sequenza di progetto a cura del membro del gruppo responsabile del caso d'uso VisualizzaContenutiCorso.

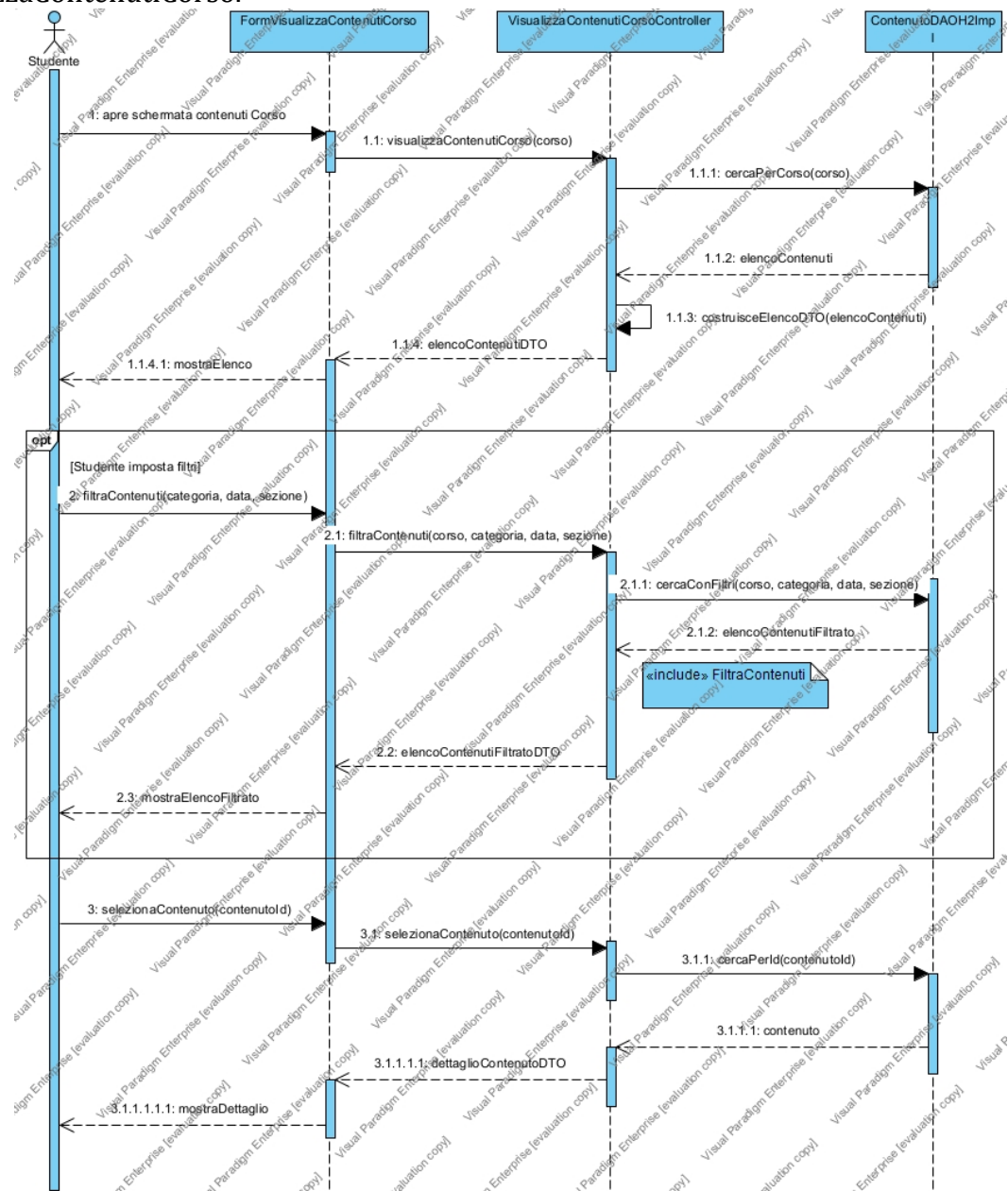


Diagramma di sequenza di progetto di VisualizzaContenutiCorso, generato in Visual Paradigm.

I sequence progettati sono stati fondamentali per la corretta implementazione dell'applicazione software ed ha fatto nascere la necessità di definire ulteriori classi, metodi e funzioni, che hanno arricchito passo dopo passo il [Diagramma delle Classi di Progettazione](#), fino ad ottenere la seguente versione finale:

Con tutti e 4 i diagrammi di sequenza di progetto disponibili, si è verificato che nessuno di essi introduce classi, attributi o metodi non già presenti nel Diagramma delle Classi di Progettazione riportato al par. 4.1: la coerenza tra i due artefatti è quindi confermata. La versione finale del

diagramma delle classi coincide con quella già mostrata al par. 4.1, che si riporta di seguito per comodità di consultazione.

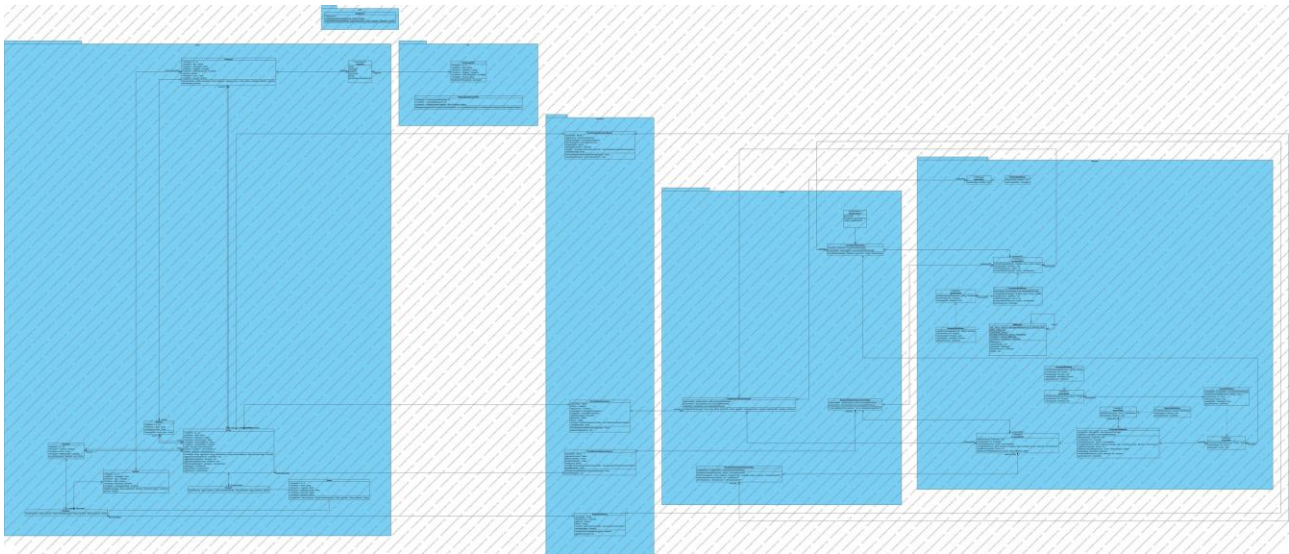


Diagramma delle Classi di Progettazione, versione finale (identico a quello del par. 4.1).

5. Implementazione

Il progetto è stato implementato in Java 11 secondo il pattern architetturale BCED (Boundary-Control-Entity-Database), con persistenza su database H2 embedded tramite il pattern DAO (par. 4.1.2). Il codice sorgente è organizzato nei package boundary, control, entity, database, oltre ai package dto e utility (estranei al pattern BCED, motivati nei par. 4.1.1 e 4.1.2.4); di seguito si descrive il contenuto di ciascun package, senza riportare il codice sorgente per intero (disponibile nel repository GitHub del progetto e nella cartella JavaProject/ della consegna).

Artefatti necessari per l'installazione e l'esecuzione (esclusi gli strumenti di sviluppo): Java Runtime Environment 11 o superiore installato sulla macchina dell'utilizzatore; il file eseguibile `materialeDidattico-1.0-SNAPSHOT.jar` (prodotto da Maven con le dipendenze incluse), avviabile con `java -jar materialeDidattico-1.0-SNAPSHOT.jar`; nessun database server da installare separatamente, poiché H2 è incluso come libreria nel jar e opera in modalità embedded (vincolo V02, par. 2.4.4): al primo avvio viene creato automaticamente il file `data/materialeDidattico.mv.db` nella cartella di esecuzione. Il diagramma di deployment che illustra questa architettura è riportato nel par. 5.6.

La documentazione del codice è prodotta con Javadoc nei commenti delle classi e dei metodi più significativi (in particolare le classi Control e la classe DBManager, par. 5.1 e 5.3).

5.1 Package Database

Il package database (`it.unina.materialeDidattico.database`) realizza il livello Database del pattern BCED secondo il pattern DAO descritto in dettaglio nel par. 4.1.2.4. Contiene la classe

DBManager, Singleton che gestisce l'unica connessione JDBC verso il database H2 embedded ed espone i metodi pubblici `getIstanza()`, `getConnessione()` e `chiudi()`; alla prima connessione esegue automaticamente lo script `schema.sql` (letto dal classpath) per creare le tabelle se non già presenti.

Contiene inoltre sette interfacce DAO (`CorsoDAO`, `DocenteDAO`, `StudenteDAO`, `SezioneDAO`, `ContenutoDAO`, `IscrizioneDAO`, `NotificaDAO`), una per ciascuna Entity persistita, con le rispettive implementazioni H2 (suffisso `H2Impl`) che usano `PreparedStatement` per prevenire SQL injection.

Eccezioni: le classi DAO non definiscono eccezioni checked custom; gli errori tecnici imprevedibili di accesso al database (es. `SQLException`) vengono intercettati e rilanciati come `RuntimeException`, per non obbligare le classi Control a gestire esplicitamente dettagli di persistenza. Gli esiti applicativi attesi e non eccezionali (es. Corso inesistente) sono invece modellati come valori di ritorno (enumerazioni) restituiti dai Controller, non come eccezioni usate per il controllo di flusso ordinario.

5.2 Package Entity

Il package entity contiene le classi del modello di dominio, corrispondenti al diagramma delle classi di analisi (par. 2.6) arricchito, a livello di progettazione, dell'attributo id (chiave primaria per la persistenza): `Utente` (astratta), `Docente` e `Studente` (che la estendono), `Corso`, `Sezione`, `Contenuto`, `Categoria` (enumerazione), `Iscrizione` (classe associativa reificata) e `Notifica`. Nessuna di queste classi accede direttamente al database: la persistenza è delegata esclusivamente alle classi DAO del package database (par. 5.1), coerentemente con le regole di dipendenza del pattern BCED.

5.3 Package Controller

Il package control contiene i quattro Use Case Controller, uno per ciascuno dei 4 casi d'uso sviluppati in dettaglio: `IscrivitiCorsoController`, `PubblicaContenutoController`, `MonitoraAndamentoCorsoController`, `VisualizzaContenutiCorsoController`. Ciascuno coordina le regole di business del proprio caso d'uso invocando le classi DAO necessarie, secondo la logica già descritta negli scenari (par. 2.5.3) e nei diagrammi di sequenza di progetto (par. 4.2).

`IscrivitiCorsoController` definisce inoltre l'enumerazione interna `EsitoIscrizione` (`SUCCESSO`, `STUDENTE_GIA_ISCRITTO`, `CORSO_INESISTENTE`), usata come tipo di ritorno per comunicare alla Boundary l'esito dell'operazione senza ricorrere a eccezioni per il controllo di flusso ordinario (vedi anche par. 6.1, dove questo metodo è usato come esempio di Control Flow Graph).

5.4 Package Boundary

Il package boundary contiene le quattro classi grafiche, una per caso d'uso (`FormIscrivitiCorso`, `FormPubblicaContenuto`, `FormMonitoraAndamentoCorso`, `FormVisualizzaContenutiCorso`), tutte realizzate in Java Swing (vincolo V03, par. 2.4.4) come sottoclassi di `JFrame`. Prima di invocare il proprio Controller, ciascuna Form esegue la validazione di formato dei dati inseriti tramite la classe di utilità `Validazione` (package utility, estraneo al pattern BCED, par. 4.1.1): solo se questo controllo ha esito positivo la richiesta viene inoltrata al Controller, che si occupa esclusivamente delle regole di business che richiedono accesso al database.

5.5 Package DTO

L'introduzione di tale package, estraneo al pattern BCED, nasce dall'esigenza di mostrare sulla GUI collezioni di elementi.

Da questo punto di vista, il problema principale è proprio quello che, qualora una determinata chiamata a funzione restituisse alla GUI un elenco di entity, questa, per poterlo visualizzare correttamente a video, dovrebbe conoscere di fatto la struttura interna di tale classe Entity, ma ciò porterebbe con sé un accoppiamento troppo elevato e quindi una chiara violazione dei vincoli del pattern a livelli adottato.

Si introduce allora il concetto di **Data Transfer Object (DTO)**, un oggetto in grado di trasportare dati tra processi (nel caso in oggetto tra livelli). Le classi DTO hanno tipicamente una struttura che rispecchia quella dell'entity di cui vanno a supporto, in particolare gli attributi coincidono con quelli dell'entity che si intendono visualizzare a schermo.

5.6 Diagramma di Deployment

I diagrammi di deployment sono utilizzati per mostrare l'architettura fisica del sistema software realizzato; sono particolarmente utili per valutare, durante lo sviluppo, come un'applicazione si distribuisce tra le varie macchine. Nel nostro caso l'architettura è volutamente semplice: un unico nodo (il PC dell'utilizzatore) esegue sia l'applicazione Java Swing sia il motore del database H2 in modalità embedded, senza alcun server di persistenza separato da installare o configurare (vincolo V02, par. 2.4.4).

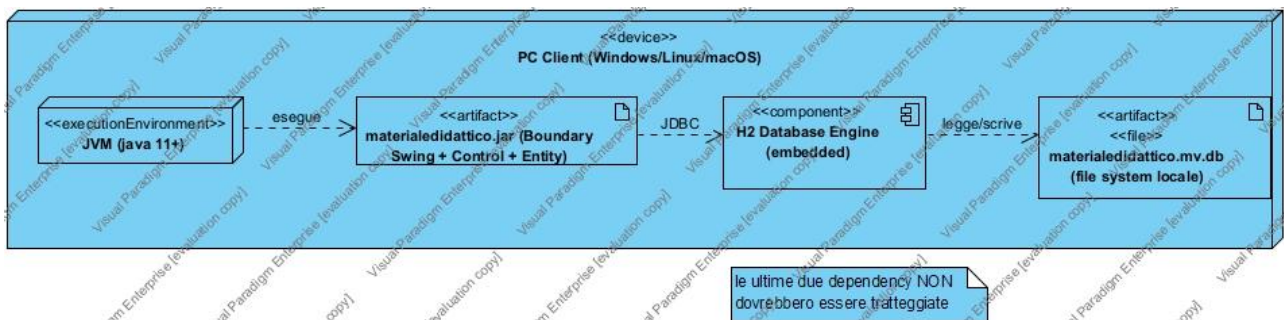


Diagramma di deployment: nodo unico PC Client, JVM, jar applicativo e database H2 embedded.

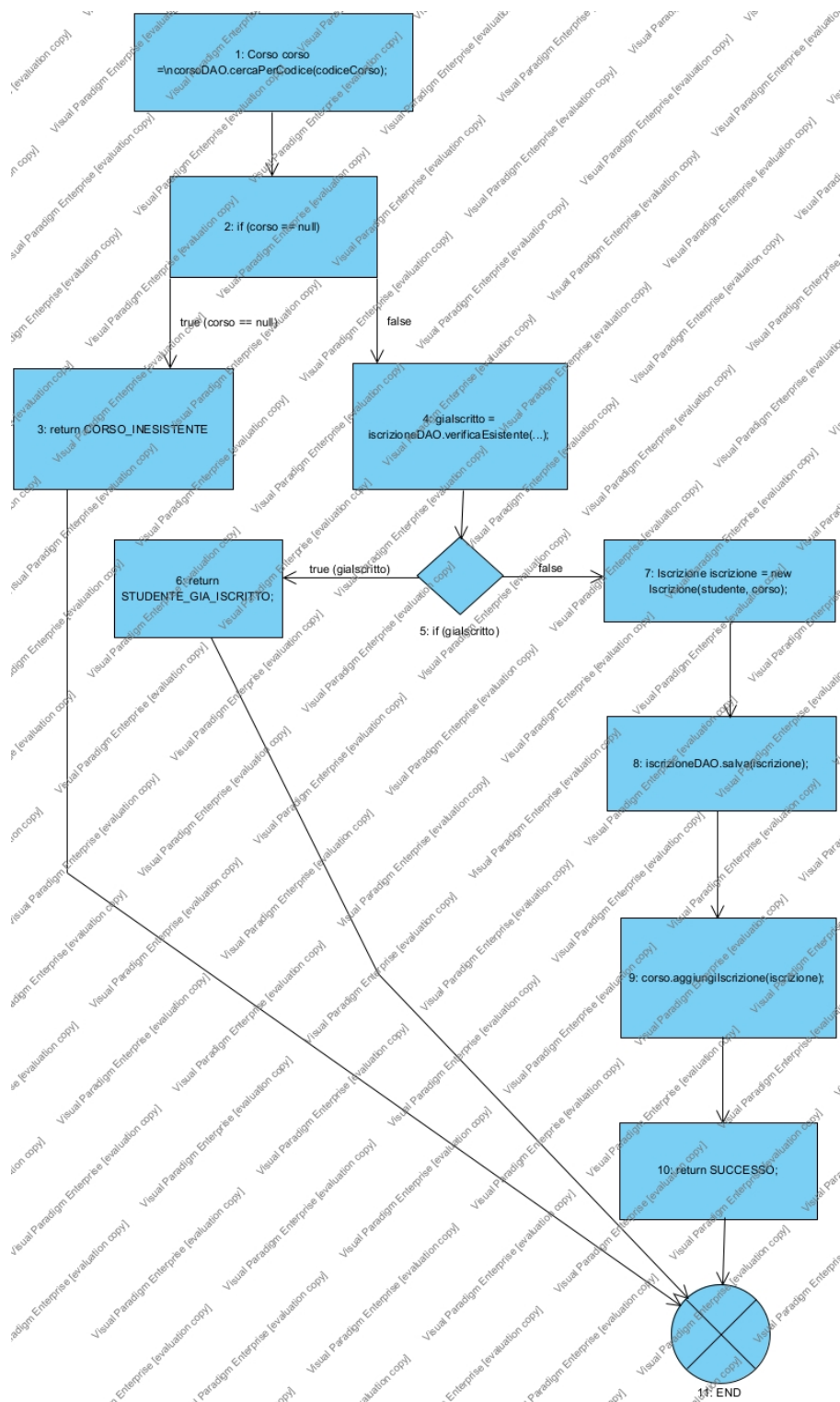
6. Testing

6.1 Test Strutturale

6.1.1 Complessità ciclomatica

Come metodo non banale su cui costruire il Control Flow Graph si è scelto `IscrivitiCorsoController.iscrivitiCorso` (package `control`, par. 5.3), già presentato come esempio nel par. 4.1.1: è il metodo che implementa le regole di business del caso d'uso `IscrivitiCorso` (UC6), con due decisioni indipendenti che corrispondono esattamente ai tre esiti dell'enumerazione `EsitoIscrizione` e ai tre casi di test già individuati nel Piano di test funzionale (par. 3.1).

```
public EsitoIscrizione iscrivitiCorso(Studiante studente, String codiceCorso) {  
    Corso corso = corsoDAO.cercaPerCodice(codiceCorso);           (1)  
    if (corso == null) {                                         (2)  
        return EsitoIscrizione.CORSO_INESISTENTE;              (3)  
    }  
    boolean giaIscritto = iscrizioneDAO.verificaEsistente(studente, corso); (4)  
    if (giaIscritto) {                                           (5)  
        return EsitoIscrizione.STUDENTE_GIA_ISCRITTO;          (6)  
    }  
    Iscrizione iscrizione = new Iscrizione(studente, corso);     (7)  
    iscrizioneDAO.salva(iscrizione);                             (8)  
    corso.aggiungiIscrizione(iscrizione);                       (9)  
    return EsitoIscrizione.SUCCESSO;                           (10)  
}
```



Control Flow Graph di IscrivitiCorsoController.iscrivitiCorso (nodi numerati come nel listato; nodo 11 = uscita unica del metodo).

NUMERO CICLOMATICO:

numero di regioni chiuse del grafo + 1 = 2 + 1 = 3

numero di nodi predicati (2, 5) + 1 = 2 + 1 = 3

archi - # nodi + 2 = (12 - 11) + 2 = 3

CAMMINI LINEARMENTE INDIPENDENTI:

- 1) 1-2-3-11 (corso == null, ramo CORSO_INESISTENTE)
- 2) 1-2-4-5-6-11 (corso trovato, Studente già iscritto, ramo STUDENTE_GIA_ISCRITTO)
- 3) 1-2-4-5-7-8-9-10-11 (corso trovato, Studente non ancora iscritto, ramo SUCCESSO)

I 3 cammini corrispondono esattamente ai 3 casi di test già progettati per `IscrivitiCorso` nel Piano di test funzionale (par. 3.1, Category Partition Testing): si riutilizzano quegli stessi identificativi invece di duplicare la tabella, dato che input, pre-condizioni e output attesi sono già descritti lì.

Cammino	Condizione	Caso di test (par. 3.1)	Esito atteso
1-2-3-11	corso == null (Corso inesistente)	TC5	CORSO_INESISTENTE
1-2-4-5-6-11	corso trovato, <code>giàiscritto == true</code>	TC6	STUDENTE_GIA_ISCRITTO
1-2-4-5-7-8-9-10-11	corso trovato, <code>giàiscritto == false</code>	TC1	SUCCESSO

6.2 JUnit – Test di Unità

Si riportano alcuni casi di test JUnit 5 (Jupiter, dipendenza aggiunta al `pom.xml`) sulla classe `Validazione` (package `utility`, par. 4.1.1 e 5.4), che raccoglie i controlli di formato usati dalle Boundary prima di invocare il Controller: è una classe pura, senza dipendenze dal database, quindi adatta a test di unità automatizzati che non richiedono un ambiente H2 attivo. I casi scelti ricalcano le classi di equivalenza [ERROR] già individuate nel Piano di test funzionale (par. 3.1 e 3.2). La classe di test completa è in `src/test/java/it/unina/materialedidattico/utility/ValidazioneTest.java`.

```
class ValidazioneTest {

    @Test
    void codiceCorsoValidoRestituisceTrue() {
        assertTrue(Validazione.verificaCodiceCorso("CINF2024A"));
    }

    @Test
    void codiceCorsoVuotoRestituisceFalse() {
        assertFalse(Validazione.verificaCodiceCorso(""));
    }

    @Test
    void datiPubblicazioneCompletiRestituisconoTrue() {
        assertTrue(Validazione.verificaDatiPubblicazione(
            "Slide Lezione 1", "Introduzione al corso", Categoria.SLIDE));
    }

    @Test
    void titoloVuotoRestituisceFalse() {
        assertFalse(Validazione.verificaDatiPubblicazione(
            "", "Introduzione", Categoria.SLIDE));
    }

    @Test
    void categoriaNonSpecificataRestituisceFalse() {
        assertFalse(Validazione.verificaDatiPubblicazione(
            "Slide Lezione 1", "Introduzione", null));
    }
}
```

```
}  
// ... (altri 5 casi in ValidazioneTest.java: codice solo spazi/null,  
//      descrizione vuota/solo spazi, titolo null)  
}
```

L'esecuzione di `mvn test` in IntelliJ ha confermato tutti e 10 i casi riportati: risultato 10/10 PASS, coerente con gli esiti attesi.

6.3 Test funzionale

Segue una descrizione in forma tabellare dei risultati dell'esecuzione dei test funzionali precedentemente pianificati.

I casi di test sono stati eseguiti sull'applicazione completa, con il database H2 popolato tramite `sql/seed_data.sql` e le quattro GUI Swing funzionanti. Durante l'esecuzione è emerso un difetto in `ContenutoDAOH2Impl`: la data di pubblicazione letta dal database veniva sovrascritta con la data corrente invece di riportare quella realmente memorizzata; il costruttore di `Contenuto` è stato quindi affiancato da un metodo `setDataPubblicazione`, usato da `ContenutoDAOH2Impl.mappaRiga` per ripristinare il valore corretto. Per due casi (`MonitoraAndamentoCorso TC3` e `VisualizzaContenutiCorso TC4`) il controllo previsto richiederebbe un sistema di autenticazione, volutamente escluso dai 4 casi d'uso sviluppati in dettaglio: sono quindi segnalati come non applicabili in questa versione, anziché forzare un esito che il sistema non è progettato per produrre.

6.3.1 IscrivitiCorso (rif. par. 3.1)

TC ID	Esito	Output ottenuti / Post-condizioni ottenute / Note
TC1	PASS	Testato con il codice CINF2024B, dato che lo Studente demo (Giovanni Aliperta) risultava già iscritto a CINF2024A: stessa classe di equivalenza (Corso esistente, Studente non ancora iscritto). L'iscrizione è andata a buon fine; un tentativo successivo con lo stesso codice ha correttamente restituito "già iscritto", confermando indirettamente la riuscita del primo.
TC2	PASS	Codice vuoto: <code>Validazione.verificaCodiceCorso("") = false</code> , Controller mai invocato. Verificato realmente (par. 6.2).
TC3	PASS	Codice con simboli ('CI@NF#'): <code>Validazione.verificaCodiceCorso(...)</code> = false. Verificato realmente.
TC4	PASS	Codice troppo lungo (17 caratteri): <code>Validazione.verificaCodiceCorso(...)</code> = false. Verificato realmente.
TC5	PASS	Testato con codice "ZZZZ9999": il sistema ha correttamente mostrato il messaggio "Codice Corso non valido: nessun Corso trovato". Nessuna Iscrizione creata.
TC6	PASS	Testato con lo Studente demo (Giovanni Aliperta), già iscritto al Corso: il sistema ha correttamente mostrato il messaggio "Risulti già iscritto a questo Corso". Nessuna nuova Iscrizione creata.



6.3.2 PubblicaContenuto (rif. par. 3.2)

TC ID	Esito	Output ottenuti / Post-condizioni ottenute / Note
TC1	PASS	Contenuto "Slide Lezione 1" creato e pubblicato immediatamente; comparso correttamente nell'elenco dei Contenuti del Corso (verificato anche da VisualizzaContenutiCorso, par. 6.3.4) e conteggiato nel riepilogo di MonitoraAndamentoCorso (par. 6.3.3).
TC2	PASS	Contenuto "Esercizi Modulo 2" creato con Sezione "Modulo 2 - Progettazione BCED" e casella "pubblica subito" deselezionata: correttamente assente dall'elenco visibile agli Studenti (par. 6.3.4) e non conteggiato tra i Contenuti pubblicati (par. 6.3.3).
TC3	PASS	Titolo vuoto: Validazione.verificaDatiPubblicazione("", ...) = false. Verificato realmente.
TC4	PASS	Titolo > 100 caratteri: rilevato dal nuovo controllo di lunghezza aggiunto a Validazione (assente nella versione precedente, corretto durante questa fase di testing). Verificato realmente.
TC5	PASS	Descrizione vuota: Validazione.verificaDatiPubblicazione(..., "", ...) = false. Verificato realmente.
TC6	PASS	Descrizione > 500 caratteri: rilevato dal nuovo controllo di lunghezza aggiunto a Validazione. Verificato realmente.
TC7	PASS	Categoria non specificata (null): Validazione.verificaDatiPubblicazione(..., null) = false. Verificato realmente.
TC8	PASS (per costruzione)	Il combobox Categoria nella Form mostra solo i valori dell'enumerazione Categoria: non è possibile selezionare un valore non valido dall'interfaccia. La classe di equivalenza è quindi soddisfatta per costruzione, non tramite un test manuale.
TC9	PASS (per costruzione)	Il combobox Sezione nella Form mostra solo le Sezioni del Corso corrente (SezioneDAO.cercaPerCorso): non è possibile selezionare una Sezione di un altro Corso dall'interfaccia. La classe di equivalenza è quindi soddisfatta per costruzione.



6.3.3 MonitoraAndamentoCorso (rif. par. 3.3)

TC ID	Esito	Output ottenuti / Post-condizioni ottenute / Note
TC1	PASS	Riepilogo mostrato: 4 Contenuti pubblicati, 2 Studenti iscritti, distribuzione SLIDE: 3, DISPENSE: 1 (numeri cumulativi dopo l'esecuzione dei test di PubblicaContenuto, par. 6.3.2, che hanno aggiunto un nuovo Contenuto pubblicato al Corso).
TC2	VERIFICATO (logica)	Nei dati di esempio non è presente un Corso privo di Contenuti e Studenti, quindi il caso non è stato eseguito dalla GUI. La logica è comunque banale (le query COUNT restituiscono naturalmente 0 su un risultato vuoto) ed è stata verificata per ispezione del codice di ContenutoDAO e IscrizioneDAO.
TC3	N/A	Non applicabile in questa versione: il controllo di titolarità richiederebbe un sistema di autenticazione, volutamente escluso dai 4 casi d'uso sviluppati in dettaglio (par. 1). MonitoraAndamentoCorsoController non effettua oggi questo controllo.

6.3.4 VisualizzaContenutiCorso (rif. par. 3.4)

TC ID	Esito	Output ottenuti / Post-condizioni ottenute / Note
TC1	PASS	Elenco completo mostrato correttamente (4 Contenuti pubblicati nel Corso). Selezionando "Dispensa Requisiti" il pannello di dettaglio mostra correttamente categoria, sezione, data di pubblicazione e descrizione.
TC2	PASS	Filtro per categoria SLIDE applicato: elenco filtrato correttamente a 3 Contenuti di categoria SLIDE.
TC3	PASS	Filtro per categoria AVVISI applicato: elenco correttamente vuoto, nessun avviso pubblicato nel Corso.
TC4	N/A	Non applicabile in questa versione: il controllo di iscrizione richiederebbe un sistema di autenticazione, volutamente escluso dai 4 casi d'uso sviluppati in dettaglio (par. 1).



		VisualizzaContenutiCorsoController non effettua oggi questo controllo.
TC5	PASS (per costruzione)	La Form permette di selezionare solo Contenuti realmente presenti nell'elenco caricato dal Corso: un identificativo inesistente non è raggiungibile dall'interazione normale con l'interfaccia.

